2026

Relatório de Integração de Dados de Acidentes Rodoviários de Londres

AGOSTO DE 2025

Universidade Politécnica do Cávado e do Ave
Engenharia de Sistemas Informáticos - 3º ano
Integração de Sistemas de Informação
David Faria - 31517
Rafael Costa - 31524

Índice

1-Enquadramento *3*

2-Problema *3*

3-Estratégia utilizada *4*

3.1-Fontes de Dados **5**

3.2-Delimitação e Preparação dos Dados **6**

3.2.1-Dataset Collisions 6

3.2.2-Dataset Vehicles 7

3.3-Arquitetura Geral da Solução **8**

4- Transformações *9*

4.1- Implementação com Pentaho Kettle (Aluno 31524) **9**

4.1.1- Limpeza e Validação dos Dados 10

4.1.2- Processamento dos Dados 11

4.1.3- Preparação dos Dados Meteorológicos 13

4.1.4- Integração dos Dados 14

4.1.5- Carregamento de dados no SQL Server 15

4.1.6- Geração e Exportação das Estatísticas 18

4.1.7 Job 23

4.1.8 Visualização dos Resultados com Node-Red 24

4.1.9 Vídeo de Demonstração 26

4.2 - Implementação com KNIME (aluno 31517) **27**

4.2.1 - Limpeza e Validação dos Dados 27

4.2.2 - Processamento dos Dados 30

4.2.3 - Preparação dos Dados Meteorológicos 32

4.2.4 - Integração dos Dados 33

4.2.5 - Carregamento dos Dados no PostgreSQL 34

4.2.6 - Exportação dos Resultados e Envio de Notificações 36

4.2.7 - Monitorização dos Logs de Ligação ao PostgreSQL 41

4.2.8 - Visualização dos Resultados com Node-Red 44

4.2.9 – Vídeo de demonstração 46

5- Conclusão e Trabalhos Futuros *47*

6- Referências Bibliográficas *48*

Índice de Figuras

Figura 1: Filtragem do Dataset Collisions 7
Figura 2: Filtragem do Dataset Vehicles 8
Figura 3: Arquitetura Geral 9
Figura 4: Limpeza e validação de Collisions no Kettle 10
Figura 5: Limpeza e validação de Vehicles no Kettle 11
Figura 6: Processamento de Vehicles Kettle 12
Figura 7: Processamento de Collisions Kettle 12
Figura 8: Preparação e recolha de dados meteorológicos 14
Figura 9: Integração dos acidentes com a meteorologia 15
Figura 10: Estrutura inicial da base de dados AcidentesLondresDB 16
Figura 11: Carregamento dos acidentes no SQL Server 16
Figura 12: Carregamento dos veículos no SQL Server. 17
Figura 13: Resultados do carregamento dos dados nas tabelas Acidentes e Veiculos 17
Figura 14: Geração e exportação das estatísticas Ketle 18
Figura 15: Job principal da automatização do processo ETL 23
Figura 16: Email Enviado 24
Figura 17: Fluxo de leitura e preparação dos dados no Node-RED 25
Figura 18: Dashboard1 25
Figura 19:Dashboard2 26
Figura 20- Workflow de limpeza e validação de Collisions no KNIME 28
Figura 21- Workflow de limpeza e validação de Vehicles no KNIME 29
Figura 22- Workflow de processamento de Vehicles no KNIME 30
Figura 23-Workflow de processamento de Collisions no KNIME 31
Figura 24 - Workflow de preparação dos dados meteorológicos no KNIME 32
Figura 25 - Workflow de integração dos acidentes com os dados meteorológicos no KNIME 33
Figura 26 - Workflow de carregamento dos dados no PostgreSQL através do KNIME 34
Figura 27 - Verificação dos registos armazenados no PostgreSQL 35
Figura 28 - — Workflow de exportação de resultados e envio de email no KNIME. 36
Figura 29 - Execução do envio do relatório estatístico por email 40
Figura 30 - Workflow de monitorização dos logs de ligação ao PostgreSQL no KNIME 42
Figura 31 - Registos de execução do PostgreSQL Connector organizados por data decrescente 43
Figura 32 - Workflow Node-RED responsável pela leitura e preparação dos dados estatísticos 44

1-Enquadramento

A crescente disponibilidade de dados provenientes de diferentes fontes torna cada vez mais importante a utilização de ferramentas capazes de recolher, transformar e integrar informação de forma eficiente. Neste contexto, os processos de Extração, Transformação e Carregamento de dados (ETL - *Extract, Transform, Load*) desempenham um papel fundamental na preparação de informação para posterior armazenamento, análise e visualização.

O presente projeto centra-se na análise de acidentes rodoviários ocorridos em Londres, durante o mês de agosto de 2025, recorrendo à integração de diferentes fontes de dados. Para além das informações relativas aos acidentes e aos veículos envolvidos, são utilizados dados meteorológicos históricos, permitindo enriquecer a informação disponível e explorar possíveis relações entre diferentes variáveis.

O desenvolvimento do projeto foi realizado através de duas ferramentas de integração de dados, Pentaho Kettle e KNIME, permitindo implementar e explorar diferentes abordagens ao processamento de informação.

Adicionalmente, foram utilizadas tecnologias complementares para o armazenamento e apresentação dos resultados, nomeadamente o SQL Server (Kettle), PostgresSQL(KNIME) e o Node-RED, bem como mecanismos de exportação de dados e envio automático de notificações por correio eletrónico.

Desta forma, o projeto permite aplicar conhecimentos relacionados com a integração de sistemas de informação, demonstrando a utilização de diferentes tecnologias na construção de processos de tratamento e disponibilização de dados.

Todo o projeto encontra-se no **GitHub: https://github.com/Davidsvf/TP_ISI.git**

2-Problema

A análise de acidentes rodoviários constitui uma ferramenta importante para compreender os fatores associados à sinistralidade e identificar padrões que possam contribuir para uma melhor interpretação das condições em que estes ocorrem.

Apesar da existência de conjuntos de dados públicos sobre acidentes rodoviários, a informação encontra-se frequentemente distribuída por diferentes fontes, com estruturas e formatos distintos. Esta fragmentação dificulta a realização de

análises que relacionem as características dos acidentes, os veículos envolvidos e as condições meteorológicas registadas no momento da ocorrência.

No caso dos acidentes rodoviários em Londres, os dados relativos às colisões e aos veículos encontram-se organizados em conjuntos de dados separados. Por outro lado, a informação meteorológica não está diretamente associada a esses registos, sendo necessário obtê-la através de uma fonte externa e estabelecer uma correspondência com as datas e localizações dos acidentes.

Adicionalmente, os dados originais podem apresentar valores em falta, registos duplicados, códigos numéricos pouco intuitivos ou informações que necessitam de validação e transformação antes de poderem ser utilizadas numa análise.

Perante estas dificuldades, surge a necessidade de desenvolver um processo que permita extrair, limpar, transformar e integrar os diferentes conjuntos de dados, garantindo uma estrutura consistente e adequada à sua posterior utilização.

Assim, o problema central deste projeto consiste na integração de dados rodoviários e meteorológicos provenientes de diferentes fontes, de forma a disponibilizar informação organizada que permita analisar a distribuição dos acidentes por gravidade, período do dia, características dos condutores e condições meteorológicas.

Pretende-se, deste modo, transformar dados inicialmente dispersos em informação estruturada e acessível, facilitando a identificação de padrões e a visualização dos resultados obtidos.

3-Estratégia utilizada

A estratégia adotada para o desenvolvimento deste projeto baseou-se na implementação de processos de Extração, Transformação e Carregamento de dados (ETL), recorrendo a duas ferramentas distintas: Pentaho Data Integration (Kettle) e KNIME Analytics Platform.

O trabalho foi organizado em três fases principais. Na primeira fase, procedeu-se à recolha e preparação dos dados relativos aos acidentes rodoviários e aos veículos envolvidos, provenientes de ficheiros CSV disponibilizados pelo sistema STATS19. Complementarmente, foram obtidos dados meteorológicos históricos através da API REST Open-Meteo, em formato JSON.

Na segunda fase, foram realizadas operações de limpeza, validação, transformação e integração dos dados. Estas operações incluíram a identificação e tratamento de valores inválidos, remoção de registos duplicados, normalização de atributos, conversão de códigos em descrições textuais, agrupamentos e cruzamento de informação entre diferentes fontes.

Na terceira fase, procedeu-se ao armazenamento dos resultados em bases de dados relacionais, utilizando SQL Server na implementação desenvolvida em Pentaho Kettle e PostgreSQL na implementação realizada em KNIME. Foram ainda desenvolvidos mecanismos de exportação de dados, produção de estatísticas, envio de relatórios por correio eletrónico e visualização dos resultados através de dashboards Node-RED.

Nos capítulos seguintes são apresentadas as fontes utilizadas, a arquitetura geral da solução e as transformações implementadas em cada ferramenta.

3.1-Fontes de Dados

Para o desenvolvimento do projeto, foram utilizadas diferentes fontes de dados, de forma a permitir a integração de informação relacionadas com os acidentes rodoviários, os veículos envolvidos e as condições meteorológicas.

A principal fonte de informação corresponde aos dados oficiais de sinistralidade rodoviária do Reino Unido, disponibilizados através do sistema STATS19. Desta fonte foram selecionados dois conjuntos de dados, em formato CSV:

- **Collisions**: contém informações relativas às ocorrências rodoviárias, nomeadamente a data e hora dos acidentes, localização geográfica, gravidade, número de vítimas e características das condições rodoviárias.
- **Vehicles:** contém informações relativas aos veículos envolvidos nos acidentes, incluindo o tipo de veículo, as manobras realizadas e características dos condutores, como a idade e o sexo.

Os dois conjuntos de dados encontram-se relacionados através do atributo collision_index, que permite identificar os veículos associados a cada acidente

Como terceira fonte de informação, foi utilizada a API histórica Open-Meteo, que permite consultar dados meteorológicos de períodos anteriores através de pedidos HTTP. Neste projeto, foram recolhidas informações relativas à temperatura, precipitação, velocidade do vento e condições meteorológicas.

A utilização desta API permitiu complementar os dados rodoviários com informação meteorológica correspondente ao período e às localizações em análise.

A combinação destas três fontes constituiu a base para o desenvolvimento do processo ETL, permitindo realizar análises que relacionam as características dos acidentes, dos veículos e dos condutores com as condições meteorológicas registadas.

Fonte	Formato	Informação
STATS19 - Collisions	CSV	Dados dos acidentes

STATS19 - Vehicles	CSV	Dados dos Veículos e condutores
Open-Meteo	JSON (API REST)	Dados meteorológicos

Tabela 1: Fontes de Dados

3.2-Delimitação e Preparação dos Dados

Os conjuntos de dados originais apresentavam um volume elevado de registos e uma grande quantidade de atributos, muitos dos quais não eram necessários para o estudo. Assim, foi realizada uma preparação inicial dos datasets Collisions e Vehicles, com o objetivo de reduzir a quantidade de informação e selecionar apenas os dados relevantes para a análise.

De seguida, são apresentadas as transformações realizadas na preparação de cada conjunto de dados.

3.2.1-Dataset Collisions

Na preparação do dataset *Collisions*, Figura 1, começou-se pela leitura do ficheiro CSV original. De seguida, foram aplicados filtros para selecionar apenas os acidentes ocorridos durante o mês de agosto de 2025, na região de Londres.

Posteriormente, foram removidas as colunas consideradas desnecessárias para a análise, mantendo apenas os atributos relevantes para as fases seguintes do projeto:

- collision_index;
- longitude;
- latitude;
- collision_severity;
- number_of_casualties;
- date;
- time;
- local_authority_ons_district;
- speed_limit;
- light_conditions;
- road_surface_conditions;
- urban_or_rural_area

Por fim, os dados resultantes foram exportados para um novo ficheiro CSV, denominado Collisions_Preparado.csv, que serviu de base às transformações posteriores.



Figura 1: Filtragem do Dataset Collisions

3.2.2-Dataset Vehicles

Na preparação do dataset Vehicles, Figura 2, começou-se pela leitura do ficheiro CSV original e pela seleção dos atributos considerados relevantes para a análise.

De seguida, os dados foram relacionados com o dataset Collisions previamente preparado, através do campo collision_index, permitindo identificar os veículos associados aos acidentes selecionados.

Posteriormente, foram filtrados os registos, mantendo apenas os veículos envolvidos nos acidentes ocorridos em Londres durante agosto de 2025. Foi também removida a coluna auxiliar utilizada no processo de associação assim como colunas consideradas desnecessárias para este projeto. As colunas finais escolhidas foram:

- collision_index;
- vehicle_reference;
- vehicle_type;
- vehicle_manoeuvre;
- skidding_and_overturning;
- sex_of_driver;
- age_of_driver

Por fim, os dados resultantes foram exportados para um novo ficheiro CSV, denominado Vehicles_Preparado.csv, para utilização nas transformações seguintes.

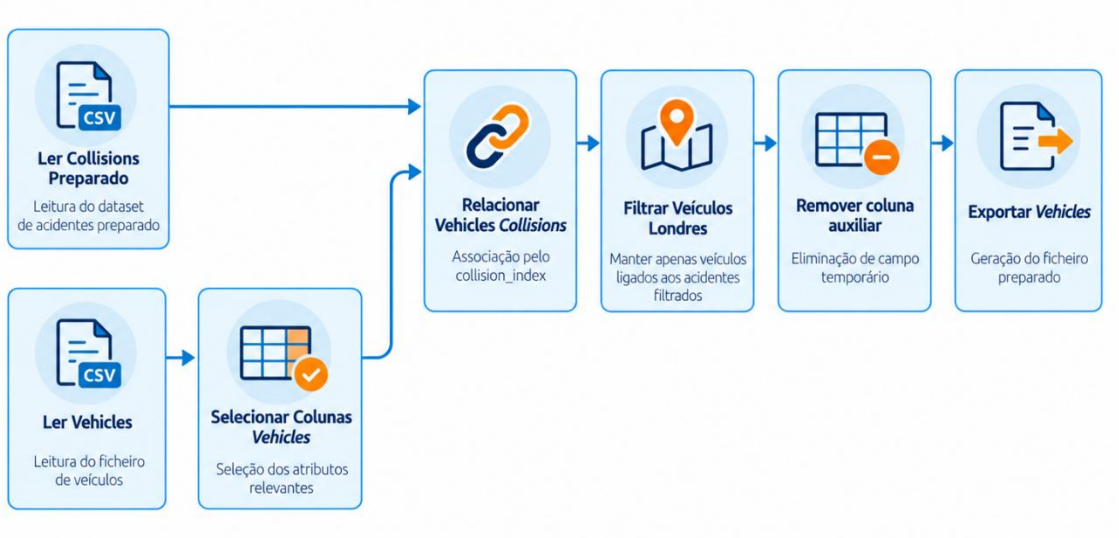


Figura 2: Filtragem do Dataset Vehicles

3.3-Arquitetura Geral da Solução

A solução desenvolvida segue uma arquitetura baseada no processo ETL (*Extract, Transform, Load*), dividida em três fases principais: extração, transformação e carregamento de dados.

Na fase de extração, foram preparados os datasets *Collisions* e *Vehicles*, juntamente com a API Open-Meteo.

Na fase de transformação, os dados são submetidos a operações de limpeza, validação, filtragem e integração, permitindo relacionar os acidentes com os veículos envolvidos e as condições meteorológicas.

Por fim, na fase de carregamento, os dados processados são armazenados numa base de dados SQL Server e PostgreSQL. São também geradas estatísticas em formato JSON, posteriormente utilizadas para o envio automático por email e para a visualização dos resultados num dashboard desenvolvido em Node-RED.

O projeto contempla implementações com Pentaho Kettle e KNIME, cujos processos serão apresentados separadamente nos capítulos seguintes.

Na Figura 3, encontra-se uma representação visual da Arquitetura geral do trabalho.

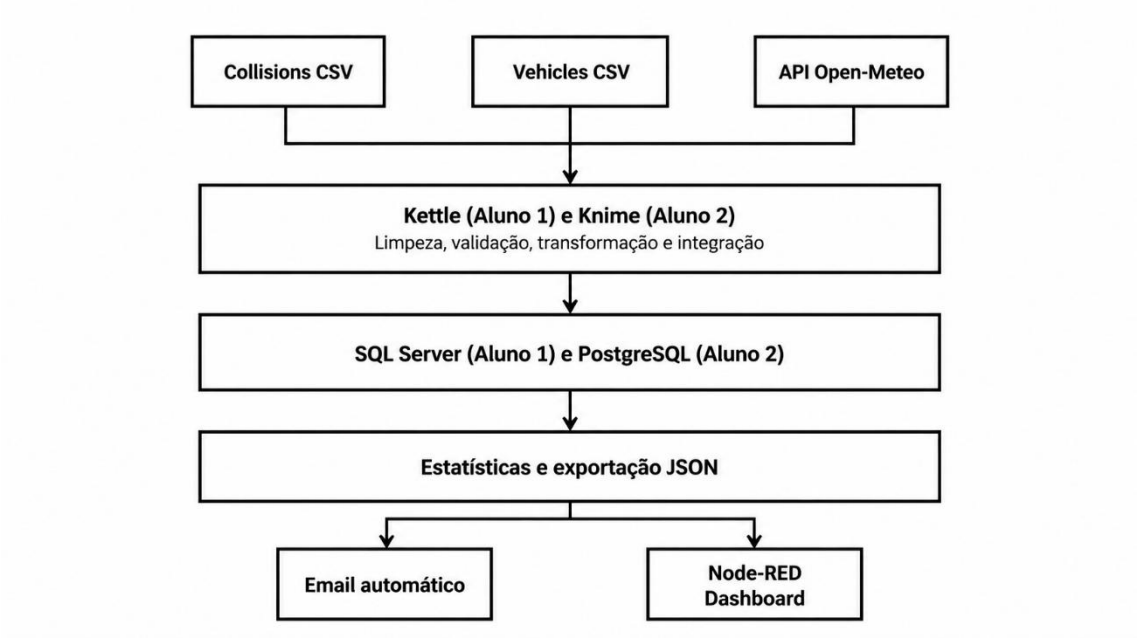


Figura 3: Arquitetura Geral

4- Transformações

Neste capítulo são apresentadas as transformações desenvolvidas ao longo do projeto, bem como os principais operadores utilizados e as respetivas funcionalidades.

De forma a organizar a apresentação do trabalho, o capítulo encontra-se dividido em dois subcapítulos: o primeiro dedicado à implementação realizada em Pentaho Kettle e o segundo à implementação desenvolvida em KNIME. Em cada um serão apresentados os diagramas das transformações e uma explicação das operações realizadas.

4.1- Implementação com Pentaho Kettle (Aluno 31524)

A implementação em Pentaho Kettle foi realizada através da ferramenta Spoon, recorrendo a diferentes transformações para executar as operações de limpeza, validação, processamento e integração dos dados.

As transformações foram organizadas de acordo com a sua finalidade, permitindo realizar cada etapa de forma independente e verificar os resultados obtidos antes de avançar para as operações seguintes.

Nos subcapítulos seguintes são apresentados os diagramas das principais transformações, acompanhados de uma descrição dos operadores utilizados e das operações realizadas.

4.1.1- Limpeza e Validação dos Dados

Após a preparação inicial dos datasets, foi realizada uma etapa de limpeza e validação, com o objetivo de garantir a consistência e a qualidade da informação utilizada nas transformações seguintes.

Este processo foi aplicado separadamente aos conjuntos de dados *Collisions* e *Vehicles*, através de operações de remoção de duplicados e filtragem de registos com valores inválidos.

Limpeza e validação de Collisions

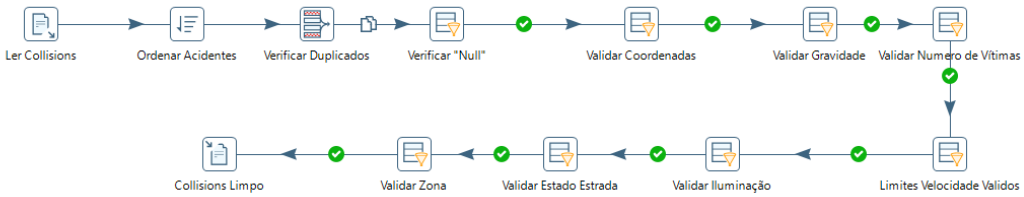


Figura 4: Limpeza e validação de Collisions no Kettle

A Figura 4 representa a transformação responsável por limpar e tratar dos dados das Collisions. A transformação inicia-se com a leitura do ficheiro Collisions_Preparado.csv, seguindo-se uma ordenação dos registos pelo campo collision_index, através do operador Sort Rows. Posteriormente, o operador Unique Rows permite identificar e remover registos duplicados com o mesmo identificador.

De seguida, são aplicados vários operadores Filter Rows, responsáveis por validar os campos considerados essenciais. Entre as verificações realizadas destacam-se a existência de identificadores válidos, os limites das coordenadas geográficas, a gravidade dos acidentes, o número de vítimas, os limites de velocidade e os códigos relativos às condições de luminosidade, ao estado da estrada e ao tipo de zona.

Os registos que não cumprem as condições definidas são excluídos do processamento, enquanto os restantes prosseguem para a etapa de exportação.

Por fim, os dados validados são guardados no ficheiro Collisions_Limpo.csv, que constitui a base para as transformações posteriores.

Limpeza e validação de Vehicles

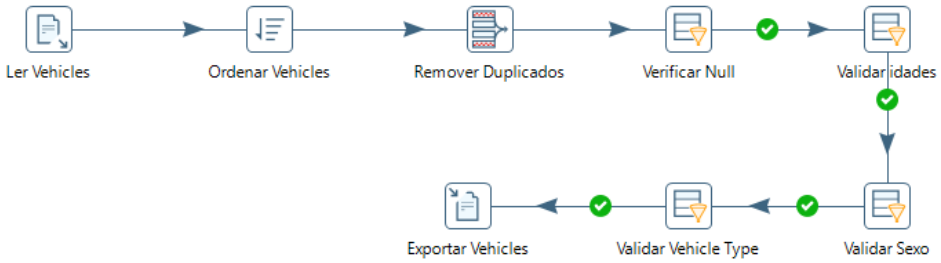


Figura 5: Limpeza e validação de Vehicles no Kettle

A Figura 5 representa a transformação responsável por limpar e tratar dos dados dos Vehicles. A transformação inicia-se com a leitura do ficheiro Vehicles_Preparado.csv, seguindo-se a ordenação dos registos pelos campos collision_index e vehicle_reference, através do operador Sort Rows.

Posteriormente, é utilizado o operador Unique Rows para remover possíveis registos duplicados, considerando a combinação destes dois campos, que permite identificar cada veículo associado a um acidente.

De seguida, são aplicados operadores Filter Rows para validar a informação, nomeadamente os identificadores dos acidentes e veículos, a idade e o sexo dos condutores e o tipo de veículo. Os registos que não cumprem as condições estabelecidas são excluídos, garantindo uma maior consistência dos dados.

Por fim, os registos validados são exportados para o ficheiro Vehicles_Limpo.csv, que será utilizado nas etapas seguintes de processamento e transformação.

4.1.2- Processamento dos Dados

Após a limpeza e validação dos datasets, foram realizadas operações de processamento com o objetivo de transformar os dados originais em informação mais organizada e fácil de interpretar.

Processamento do Dataset Vehicles

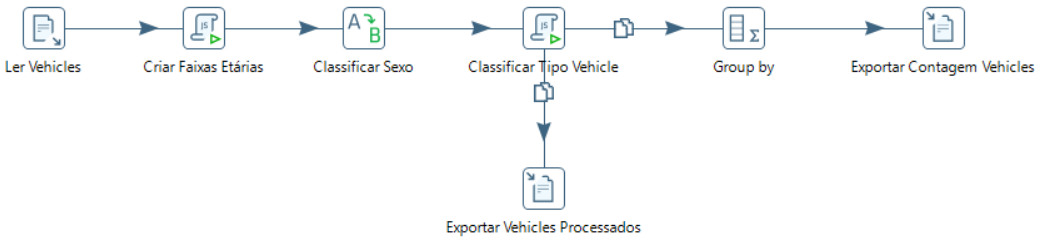


Figura 6: Processamento de Vehicles Kettle

A Figura 6 representa a transformação responsável pelo processamento dos Vehicles. A transformação inicia-se com a leitura do ficheiro Vehicles_Limpo.csv, seguindo-se a aplicação de diferentes operadores para caracterizar os veículos e os respetivos condutores.

Através do operador Modified Java Script Value, foram criadas faixas etárias com base na idade dos condutores e categorias de veículos a partir dos códigos numéricos existentes. Foi também utilizado o operador Value Mapper para converter os códigos relativos ao sexo dos condutores em descrições textuais.

Adicionalmente, foi utilizado o operador Group By para calcular o número de veículos envolvidos em cada acidente, através do agrupamento pelo campo collision_index.

Como resultado, foram gerados dois ficheiros CSV: Vehicles_Processado.csv, contendo os dados dos veículos com os novos atributos, e Veiculos_Por_Acidente.csv, com o número de veículos associados a cada acidente. Este último ficheiro será utilizado posteriormente no processamento do dataset Collisions.

Processamento do Dataset Collisions

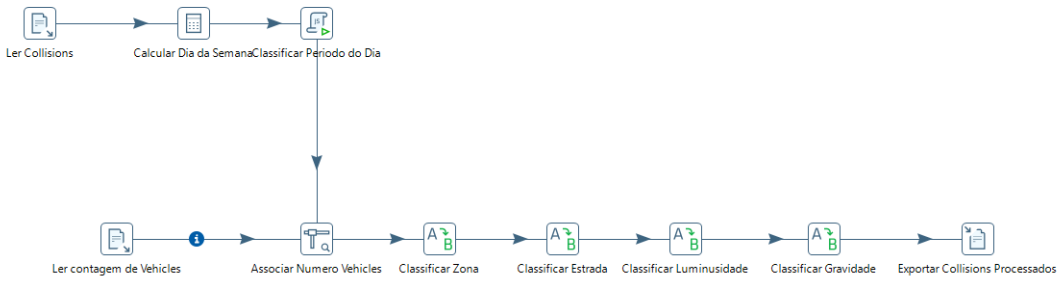


Figura 7: Processamento de Collisions Kettle

A Figura 7 representa a transformação responsável pelo processamento das Collisions. A transformação inicia-se com a leitura do ficheiro Collisions_Limpo.csv, seguindo-se a aplicação de diferentes operadores para acrescentar informação e facilitar a interpretação dos dados.

Através do operador Calculator, foi calculado o dia da semana em que ocorreu cada acidente. Posteriormente, utilizando o operador Modified Java Script Value, foi criado o atributo periodo_dia, permitindo classificar os acidentes de acordo com a hora da ocorrência.

Foi também utilizado o operador Stream Lookup para integrar o número de veículos envolvidos em cada acidente, recorrendo ao ficheiro Veiculos_Por_Acidente.csv, gerado na transformação anterior.

Adicionalmente, foram utilizados operadores Value Mapper para converter códigos numéricos em descrições textuais, nomeadamente a gravidade do acidente, o tipo de zona, as condições de luminosidade e o estado da estrada.

Por fim, os dados processados foram exportados para o ficheiro Collisions_Processado.csv, que será utilizado na fase de integração com os dados meteorológicos.

4.1.3- Preparação dos Dados Meteorológicos

Para complementar os dados dos acidentes rodoviários, foi necessário obter informação meteorológica histórica através da API Open-Meteo.

Inicialmente, considerou-se realizar um pedido à API para cada um dos 1701 acidentes. No entanto, esta abordagem implicaria um número elevado de pedidos, tornando o processo pouco eficiente.

Como alternativa, optou-se por agrupar os acidentes pelos 33 distritos de Londres, utilizando coordenadas geográficas representativas de cada distrito. Desta forma, foi possível obter os dados meteorológicos horários de todo o mês de agosto de 2025 com apenas 33 pedidos à API, em vez de 1701.

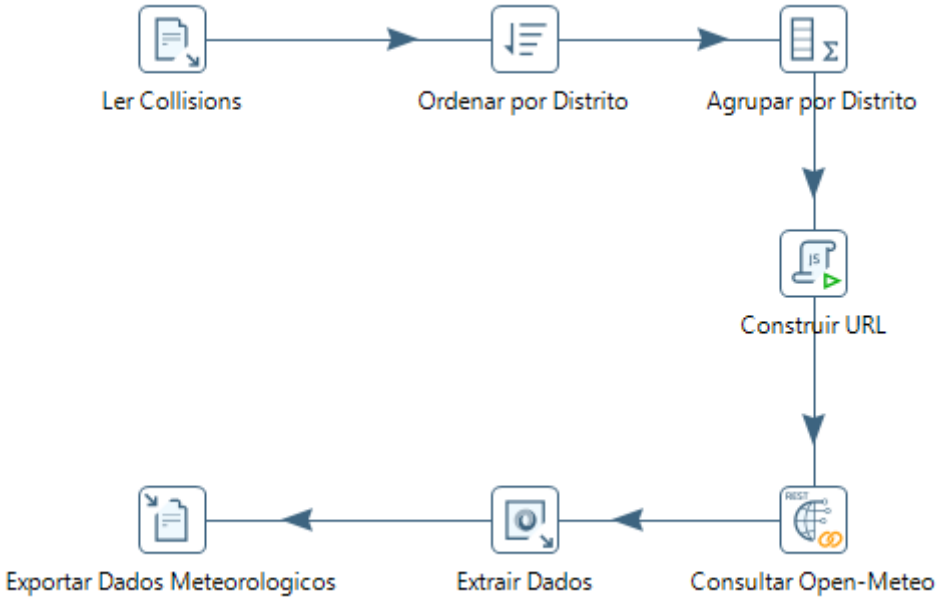


Figura 8: Preparação e recolha de dados meteorológicos

A Figura 8 representa a transformação onde são preparados e recolhidos os dados meteorológicos. A transformação inicia-se com a leitura dos dados dos acidentes, seguida de um agrupamento por distrito através do operador Group By, permitindo calcular coordenadas geográficas médias para cada zona.

Posteriormente, são construídos os endereços URL para consultar a API Open-Meteo, utilizando o operador REST Client para realizar os pedidos e obter as respostas em formato JSON.

Através do operador JSON Input, são extraídos os valores horários de temperatura, precipitação, velocidade do vento e códigos das condições meteorológicas.

Por fim, a informação recolhida é organizada e exportada para o ficheiro Meteorologia_Londres.csv, contendo 24 552 registos meteorológicos, que serão utilizados posteriormente na integração com os acidentes rodoviários.

4.1.4- Integração dos Dados

Após a preparação e o processamento dos diferentes conjuntos de dados, foi realizada a integração da informação meteorológica com os acidentes rodoviários, de forma a enriquecer os registos com as condições meteorológicas associadas a cada ocorrência.

Para estabelecer esta relação, foram utilizados dois critérios: o distrito onde ocorreu o acidente e a respetiva data e hora. Como os dados meteorológicos foram recolhidos com periodicidade horária, foi necessário ajustar a hora dos acidentes para a hora completa correspondente.

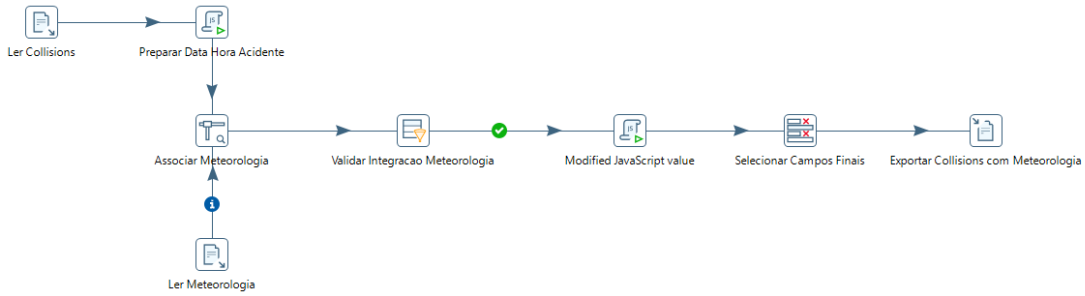


Figura 9: Integração dos acidentes com a meteorologia

A Figura 9 representa a transformação onde são integrados os dados meteorológicos com os acidentes. A transformação inicia-se com a leitura dos ficheiros Collisions_Processado.csv e Meteorologia_Londres.csv.

Através do operador Modified Java Script Value, foi criado o campo data_hora_acidente, combinando a data e a hora de cada ocorrência. Neste processo, foi utilizada uma expressão regular (Regex) para validar o formato da hora, sendo posteriormente definidos os minutos e segundos como zero. Desta forma, os horários dos acidentes foram normalizados para permitir a correspondência com os registos meteorológicos horários obtidos através da API Open-Meteo.

De seguida, foi utilizado o operador Stream Lookup para relacionar os dois conjuntos de dados através do distrito e da data e hora, acrescentando aos acidentes os valores de temperatura, precipitação, velocidade do vento e código meteorológico.

Posteriormente, foram aplicadas validações através do operador Filter Rows, garantindo que os registos integrados continham a informação meteorológica necessária.

Os códigos meteorológicos foram convertidos em descrições textuais através de um operador Modified Java Script Value, enquanto o operador Select Values permitiu selecionar os atributos relevantes para o resultado final.

Por fim, os dados integrados foram exportados para o ficheiro Acidentes_Londres_Integrados.csv, contendo 1701 registos, com informação rodoviária e meteorológica associada.

4.1.5- Carregamento de dados no SQL Server

Após a conclusão das etapas de processamento e integração, foi criada uma base de dados relacional no SQL Server, denominada AcidentesLondresDB, com o objetivo de armazenar os dados resultantes do processo ETL.

Foram criadas duas tabelas, Acidentes e Veiculos, relacionadas através do campo collision_index. Esta relação foi estabelecida através de uma chave estrangeira na tabela Veiculos, permitindo associar cada veículo ao respetivo acidente.

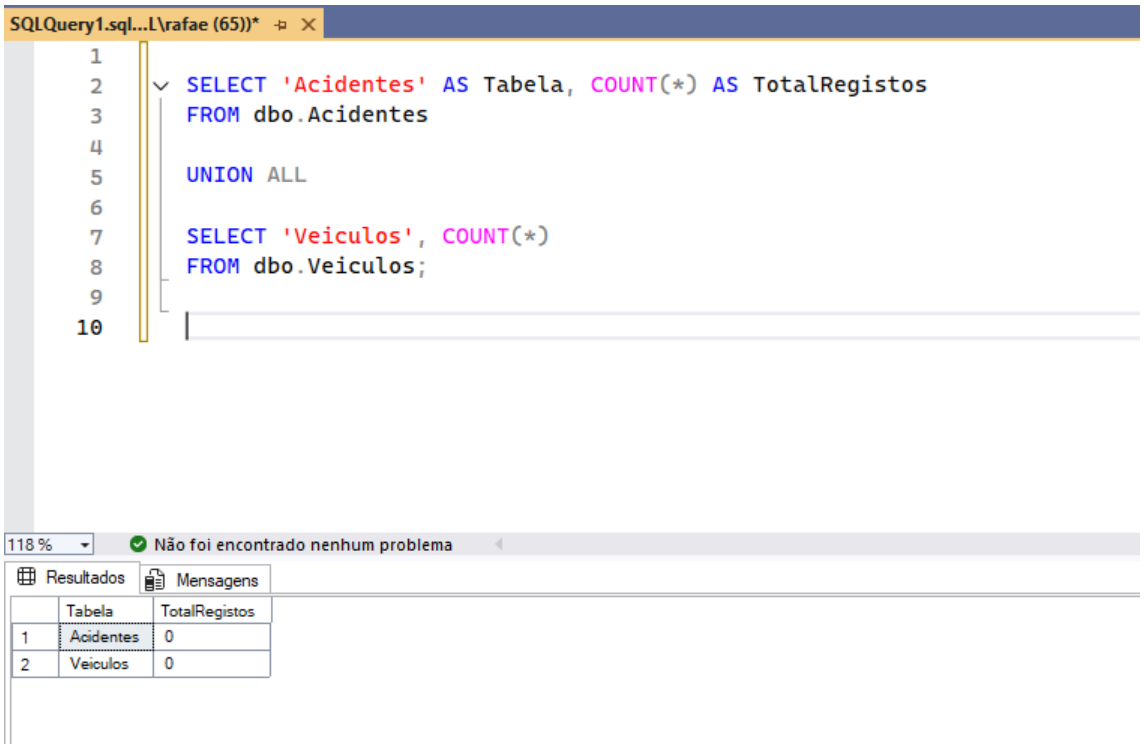


Figura 10: Estrutura inicial da base de dados AcidentesLondresDB

A Figura 10 representa a base de dados vazia, antes de qualquer transformação.

Carregamento da tabela acidentes

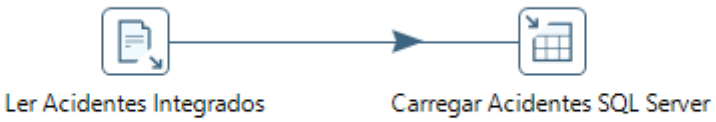


Figura 11: Carregamento dos acidentes no SQL Server

A Figura 11 representa a transformação onde os dados dos acidentes são carregados. A transformação inicia-se com a leitura do ficheiro Acidentes_Londres_Integrados.csv, que contém os dados dos acidentes enriquecidos com informação meteorológica. De seguida, através do operador Table Output, os registos são inseridos na tabela Acidentes, utilizando a ligação à base de dados previamente configurada.

Carregamento da tabela Veículos



Figura 12: Carregamento dos veículos no SQL Server.

A Figura 12 representa a transformação onde os dados dos veículos são carregados. Nesta transformação, é realizada a leitura do ficheiro Vehicles_Processado.csv, seguida da inserção dos registos na tabela Veiculos, também através do operador Table Output. A relação entre as tabelas é mantida através do campo collision_index.

Resultados do Carregamento

Após a execução das duas transformações, os dados ficaram armazenados na base de dados SQL Server, tendo sido inseridos 1701 registos de acidentes e 3120 registos de veículos.

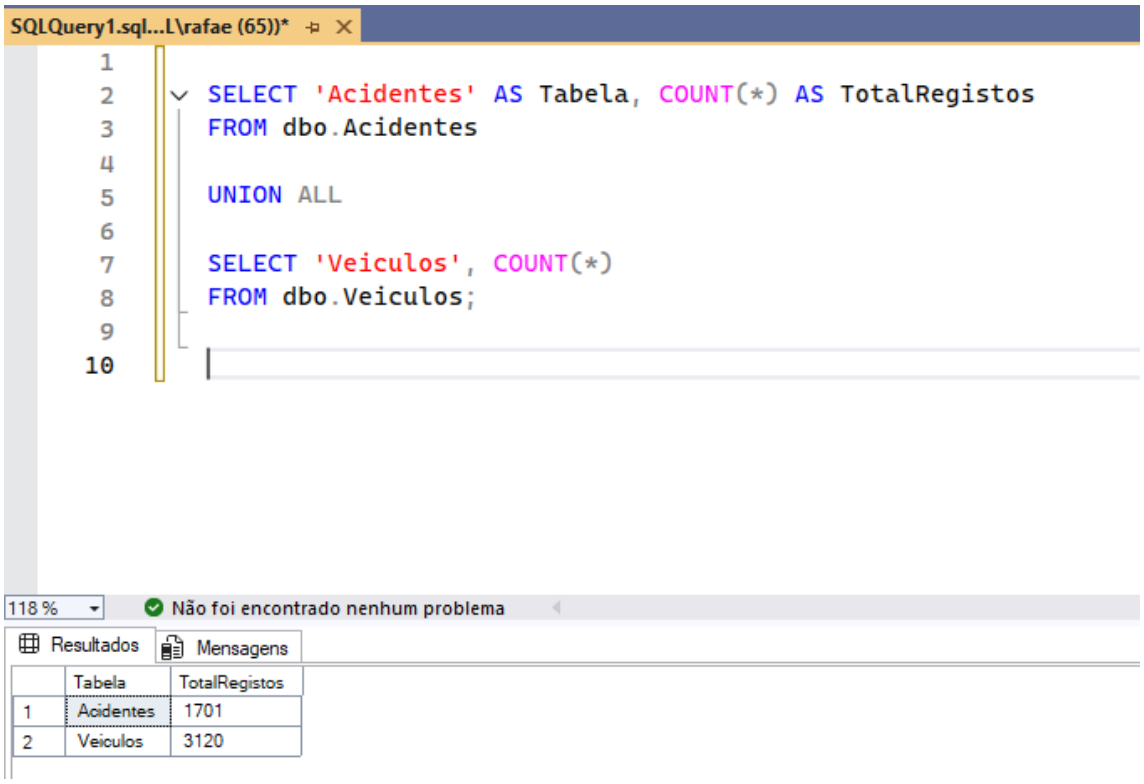


Figura 13: Resultados do carregamento dos dados nas tabelas Acidentes e Veiculos

A Figura 13 representa a base de dados já com os dados, após as duas transformações.

4.1.6- Geração e Exportação das Estatísticas

Após o carregamento dos dados no SQL Server, foi desenvolvida uma transformação destinada à obtenção de estatísticas sobre os acidentes rodoviários, os veículos envolvidos e as condições meteorológicas.

Para esse efeito, foram utilizados operadores Table Input, responsáveis pela execução de consultas SQL sobre as tabelas Acidentes e Veiculos. Estas consultas permitiram calcular diferentes indicadores e realizar agrupamentos e cruzamentos de informação.

As análises desenvolvidas incluem:

- **Estatísticas gerais:** número total de acidentes, veículos e vítimas, temperatura média e quantidade de acidentes graves e fatais.
- **Distribuição por sexo:** número de condutores por sexo.
- **Distribuição por faixa etária:** número de condutores em cada faixa etária.
- **Faixa etária e gravidade:** relação entre a idade dos condutores e a gravidade dos acidentes em que estiveram envolvidos.
- **Período do dia e gravidade:** distribuição dos acidentes pelos diferentes períodos do dia e níveis de gravidade.
- **Meteorologia e gravidade:** distribuição dos acidentes de acordo com as condições meteorológicas e a respetiva gravidade.

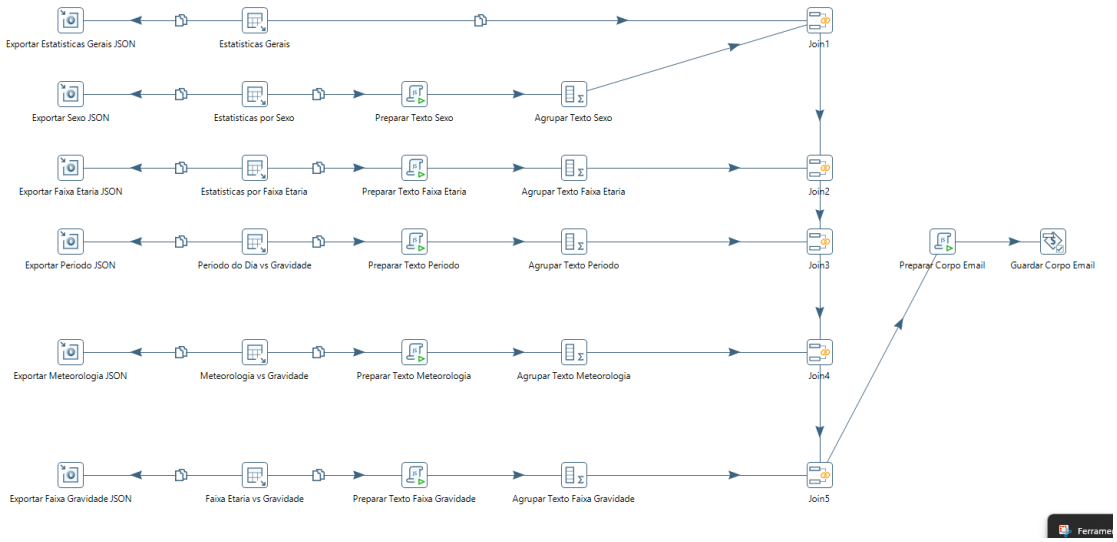


Figura 14: Geração e exportação das estatísticas Kettle

A Figura 14 represnta a transformação onde são geradas e exportadas as estatísticas. Cada consulta SQL foi ligada a um operador JSON Output, permitindo exportar os resultados para seis ficheiros JSON distintos.

Paralelamente, foram utilizados operadores Modified Java Script Value e Group By para converter os resultados das consultas em secções de texto. Posteriormente, os operadores Join Rows (cartesian product) permitiram reunir as diferentes estatísticas num único registo.

Por fim, foi construído o corpo de uma mensagem através de um operador Modified Java Script Value, sendo o texto armazenado na variável CORPO_EMAIL através do operador Set Variables.

Esta variável é posteriormente utilizada pelo Job principal para enviar automaticamente um email com um resumo das estatísticas, acompanhado dos seis ficheiros JSON em anexo.

Consultas SQL

Para a obtenção das estatísticas, foram utilizados operadores Table Input, responsáveis pela execução de diferentes consultas SQL sobre as tabelas Acidentes e Veiculos. Estas consultas recorreram a funções de agregação e operações de agrupamento e junção de dados, permitindo produzir indicadores relevantes para a análise dos acidentes rodoviários.

A Tabela 2 apresenta cada consulta e o respetivo resultado.

Função	Consulta SQL	Resultado JSON

Estatísticas Gerais	SELECT
	a.total_acidentes,
	a.total_vitimas,
	v.total_veiculos,
	a.temperatura_media,
	a.total_graves,
	a.total_fatais
	FROM
	(SELECT
	COUNT(*) AS total_acidentes,
	
	SUM(number_of_casualties) AS total_vitimas,
	
	ROUND(AVG(temperatura), 2) AS temperatura_media,
	SUM(CASE
	WHEN gravidade_acidente = 'Grave'
	THEN 1 ELSE 0
	END) AS total_graves,
	SUM(CASE
	WHEN gravidade_acidente = 'Fatal'
	THEN 1 ELSE 0
	END) AS total_fatais
	FROM dbo.Acidentes
	) a
	CROSS JOIN
		{
		"data":[{
		"total_veiculos":3120,
		"total_vitimas":1965,
		"total_fatais":7,
		"total_acidentes":1701,
		"total_graves":340,
		"temperatura_media":20.320000
		}]
		}

	(SELECT COUNT(*) AS total_veiculos FROM dbo.Veiculos) v;	

Distribuição por sexo	SELECT
	sexo_condutor,
	COUNT(*) AS total_condutores
	FROM dbo.Veiculos
	GROUP BY sexo_condutor
	ORDER BY
	total_condutores DESC;
		{
		"data":[
		{"sexo_condutor":"Masculino ","total_condutores":1891},
		{"sexo_condutor":"Não Identificado","total_condutores":644},
		{"sexo_condutor":"Feminino ","total_condutores":483},
		{"sexo_condutor":"Desconhecido ","total_condutores":102}
		]
		}

Distribuição por Faixa Etária	SELECT
	faixa_etaria,
	COUNT(*) AS total_condutores
	FROM dbo.Veiculos
	GROUP BY faixa_etaria
	ORDER BY total_condutores
	DESC;
		{
		"data":[{
		"total_condutores":878,
		"faixa_etaria":"35-54"
		},{
		"total_condutores":709,
		"faixa_etaria":"Desconhecida"
		},{
		"total_condutores":704,
		"faixa_etaria":"25-34"
		},{
		"total_condutores":360,
		"faixa_etaria":"18-24"
		},{
		"total_condutores":274,
		"faixa_etaria":"55-64"
		},{
		"total_condutores":148,
		"faixa_etaria":"65 ou mais"
		},{
		"total_condutores":47,
		"faixa_etaria":"Menor de 18"
		}]
		}

Faixa Etária vs Gravidade	SELECT v.faixa_etaria, a.gravidade_acidente, COUNT(*) AS total_veiculos FROM dbo.Veiculos v INNER JOIN dbo.Acidentes a ON v.collision_index = a.collision_index GROUP BY v.faixa_etaria, a.gravidade_acidente ORDER BY v.faixa_etaria,	{ "data": [{"total_veiculos": 1, "faixa_etaria": "18-24", "gravidade_acidente": "Fatal"}, {"total_veiculos": 70, "faixa_etaria": "18-24", "gravidade_acidente": "Grave"}, {"total_veiculos": 289, "faixa_etaria": "18-24", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 2, "faixa_etaria": "25-34", "gravidade_acidente": "Fatal"}, {"total_veiculos": 146, "faixa_etaria": "25-34", "gravidade_acidente": "Grave"}, {"total_veiculos": 556, "faixa_etaria": "25-34", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 5, "faixa_etaria": "35-54", "gravidade_acidente": "Fatal"}, {"total_veiculos": 169, "faixa_etaria": "35-54", "gravidade_acidente": "Grave"}, {"total_veiculos": 704, "faixa_etaria": "35-54", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 3, "faixa_etaria": "55-64", "gravidade_acidente": "Fatal"}, {"total_veiculos": 64, "faixa_etaria": "55-64", "gravidade_acidente": "Grave"},

	a.gravidade_acidente;	{"total_veiculos": 207, "faixa_etaria": "55-64", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 30, "faixa_etaria": "65 ou mais", "gravidade_acidente": "Grave"}, {"total_veiculos": 118, "faixa_etaria": "65 ou mais", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 4, "faixa_etaria": "Desconhecida", "gravidade_acidente": "Fatal"}, {"total_veiculos": 113, "faixa_etaria": "Desconhecida", "gravidade_acidente": "Grave"}, {"total_veiculos": 592, "faixa_etaria": "Desconhecida", "gravidade_acidente": "Ligeiro"}, {"total_veiculos": 16, "faixa_etaria": "Menor de 18", "gravidade_acidente": "Grave"}, {"total_veiculos": 31, "faixa_etaria": "Menor de 18", "gravidade_acidente": "Ligeiro"}] }

Período do Dia vs Gravidade	SELECT
	periodo_dia,
	gravidade_acidente,
	COUNT(*) AS total_acidentes
	FROM dbo.Acidentes
	GROUP BY
	periodo_dia,
	gravidade_acidente
	ORDER BY
	periodo_dia,
	gravidade_acidente;
		{
		"data": [
		{"periodo_dia": "Madrugada", "total_acidentes": 42, "gravidade_acidente": "Grave" },
		{"periodo_dia": "Madrugada", "total_acidentes": 93, "gravidade_acidente": "Ligeiro" },
		{ "periodo_dia": "Manhã", "total_acidentes": 1, "gravidade_acidente": "Fatal" },
		{ "periodo_dia": "Manhã", "total_acidentes": 65, "gravidade_acidente": "Grave" },
		{ "periodo_dia": "Manhã", "total_acidentes": 297, "gravidade_acidente": "Ligeiro" },
		{ "periodo_dia": "Noite", "total_acidentes": 1, "gravidade_acidente": "Fatal" },
		{ "periodo_dia": "Noite", "total_acidentes": 107, "gravidade_acidente": "Grave" },
		{ "periodo_dia": "Noite", "total_acidentes": 428, "gravidade_acidente": "Ligeiro" },
		{ "periodo_dia": "Tarde", "total_acidentes": 5, "gravidade_acidente": "Fatal" },
		{ "periodo_dia": "Tarde", "total_acidentes": 126, "gravidade_acidente": "Grave" },
		{ "periodo_dia": "Tarde", "total_acidentes": 536, "gravidade_acidente": "Ligeiro" }
		]
		}

		{ "data": [{ "condicao_meteorologica": "Céu limpo", "total_acidentes": 1, "gravidade_acidente": "Fatal" }, { "condicao_meteorologica": "Céu limpo", "total_acidentes": 85, "gravidade_acidente": "Grave" },

Meteorologia vs Gravidade	SELECT condicao_meteorologica, gravidade_acidente, COUNT(*) AS total_acidentes FROM dbo.Acidentes GROUP BY condicao_meteorologica, gravidade_acidente ORDER BY condicao_meteorologica, gravidade_acidente;	{ "condicao_meteorologica": "Céu limpo", "total_acidentes": 331, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Chuva fraca", "total_acidentes": 1, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Chuva fraca", "total_acidentes": 4, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Chuva moderada", "total_acidentes": 3, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Chuva moderada", "total_acidentes": 5, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Chuvisco fraco", "total_acidentes": 1, "gravidade_acidente": "Fatal" }, { "condicao_meteorologica": "Chuvisco fraco", "total_acidentes": 30, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Chuvisco fraco", "total_acidentes": 155, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Chuvisco intenso", "total_acidentes": 4, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Chuvisco moderado", "total_acidentes": 4, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Chuvisco moderado", "total_acidentes": 15, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Encoberto", "total_acidentes": 3, "gravidade_acidente": "Fatal" }, { "condicao_meteorologica": "Encoberto", "total_acidentes": 146, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Encoberto", "total_acidentes": 599, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Maioritariamente limpo", "total_acidentes": 1, "gravidade_acidente": "Fatal" }, { "condicao_meteorologica": "Maioritariamente limpo", "total_acidentes": 42, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Maioritariamente limpo", "total_acidentes": 139, "gravidade_acidente": "Ligeiro" }, { "condicao_meteorologica": "Parcialmente nublado", "total_acidentes": 1, "gravidade_acidente": "Fatal" }, { "condicao_meteorologica": "Parcialmente nublado", "total_acidentes": 29, "gravidade_acidente": "Grave" }, { "condicao_meteorologica": "Parcialmente nublado", "total_acidentes": 102, "gravidade_acidente": "Ligeiro" }] }

Tabela 2: Consultas SQL e resultados

4.1.7 Job

Para automatizar o processo ETL, foi desenvolvido um Job principal, denominado 00_ETL_Acidentes_Londres.kjb, responsável por coordenar a execução das diferentes transformações, desde a limpeza e processamento dos dados até ao seu armazenamento e disponibilização dos resultados.

O Job foi organizado de forma sequencial, sendo cada etapa executada apenas após a conclusão bem-sucedida da anterior. Esta abordagem permite controlar o fluxo de execução e interromper o processo em caso de erro.

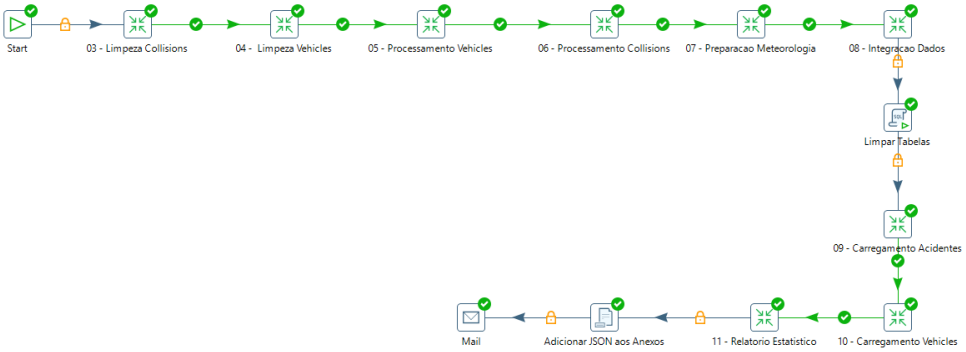


Figura 15: Job principal da automatização do processo ETL

A Figura 15: Job principal da automatização do processo ETL representa o Job principal da automatização do processo ETL. A execução inicia-se com as transformações de limpeza e validação dos datasets *Collisions* e *Vehicles*, seguindo-se o processamento dos dados, a recolha da informação meteorológica através da API Open-Meteo e a integração das diferentes fontes.

Posteriormente, é executada uma operação SQL para eliminar os registos anteriormente armazenados nas tabelas Acidentes e Veiculos, permitindo realizar novos carregamentos sem provocar conflitos com as chaves primárias. De seguida, os dados processados são carregados na base de dados SQL Server.

Após o carregamento, é executada a transformação responsável pela geração das estatísticas e pela exportação dos seis ficheiros JSON. Estes ficheiros são posteriormente adicionados aos resultados do Job para serem enviados como anexos por correio eletrónico.

Por fim, o operador Mail permite enviar automaticamente um email com o resumo das estatísticas no corpo da mensagem e os ficheiros JSON em anexo. Como é possível observar na Figura 16: Email Enviado

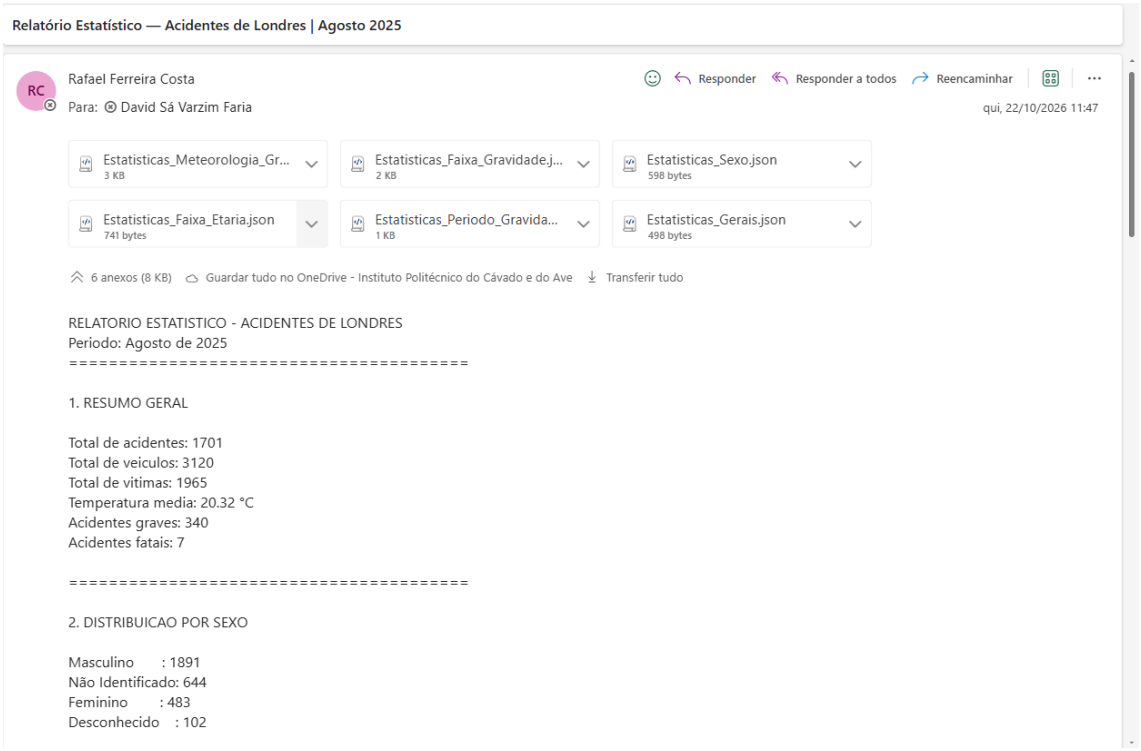


Figura 16: Email Enviado

4.1.8 Visualização dos Resultados com Node-Red

Para complementar o processo ETL, foi desenvolvido um dashboard na ferramenta Node-RED, utilizando o Dashboard 2.0, com o objetivo de apresentar os resultados obtidos de forma visual e facilitar a interpretação das estatísticas.

A integração entre o Pentaho Kettle e o Node-RED foi realizada através dos seis ficheiros JSON gerados na transformação de estatísticas, permitindo utilizar os dados processados sem necessidade de efetuar novas consultas à base de dados.

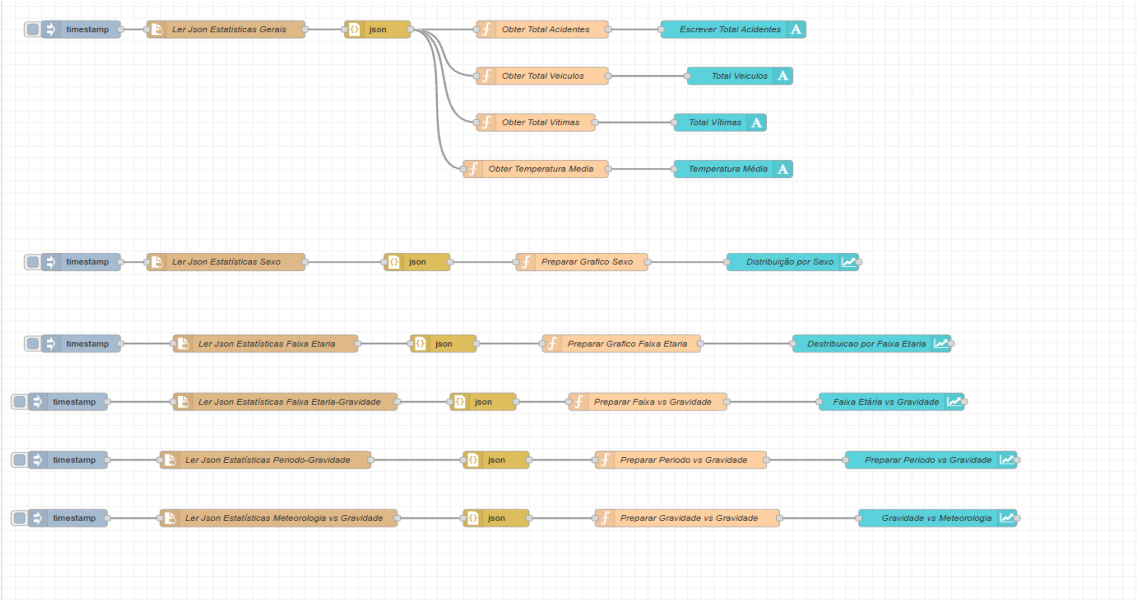


Figura 17: Fluxo de leitura e preparação dos dados no Node-RED

O fluxo utiliza operadores read file para ler os ficheiros JSON, seguindo-se operadores json para converter o conteúdo em objetos JavaScript. Posteriormente, os operadores function organizam os dados de acordo com as necessidades de cada gráfico, sendo os resultados enviados para os componentes ui-text e ui-chart do Dashboard 2.0.

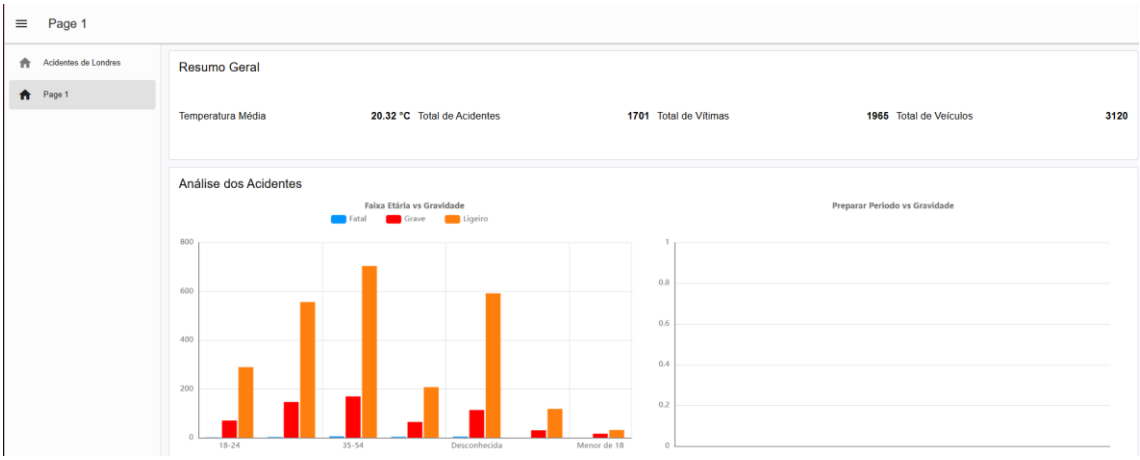


Figura 18: Dashboard1



Figura 19:Dashboard2

Na Figura 18 e Figura 19 está representado o Dashboard criado através do Node-Red. O dashboard apresenta indicadores gerais, como o número total de acidentes, veículos envolvidos, vítimas e temperatura média. Inclui também gráficos relativos ao sexo e à faixa etária dos condutores, bem como análises que relacionam a gravidade dos acidentes com a faixa etária, o período do dia e as condições meteorológicas.

Esta solução permite apresentar os resultados do processo ETL de forma organizada e intuitiva. Sempre que os ficheiros JSON são atualizados pelo Pentaho, o Node-RED pode voltar a lê-los, permitindo atualizar a informação apresentada no dashboard.

4.1.9 Vídeo de Demonstração

Foi desenvolvido um vídeo de demonstração do projeto, no qual são apresentadas as principais etapas do processo ETL, incluindo a execução das transformações, o carregamento dos dados na base de dados e a visualização dos resultados obtidos.

O vídeo pode ser consultado através do seguinte código QR.

Em caso de falha, usar o seguinte link:

https://drive.google.com/file/d/1vFIQQKoutkhPokwqFppEj7OlEXicWPcL/view?usp=sharing

4.2 - Implementação com KNIME (aluno 31517)

A implementação em KNIME foi desenvolvida através da ferramenta KNIME Analytics Platform 5.12.0, recorrendo a diferentes workflows para realizar as operações de limpeza, validação, processamento, integração, armazenamento e exportação dos dados relativos aos acidentes rodoviários em Londres.

À semelhança da implementação em Pentaho Kettle, foram utilizados os conjuntos de dados *Collisions* e *Vehicles*, previamente preparados e delimitados aos acidentes ocorridos em Londres durante agosto de 2025. Estes dados foram complementados com informação meteorológica histórica, obtida através da API REST Open-Meteo.

A solução foi organizada em workflows independentes, permitindo executar, testar e validar cada etapa separadamente. Esta abordagem modular facilita a identificação de eventuais erros, a análise dos resultados intermédios e a manutenção do processo ETL.

Foram desenvolvidos oito workflows principais, apresentados nas secções seguintes, seguindo a ordem lógica de processamento dos dados.

4.2.1 - Limpeza e Validação dos Dados

A primeira etapa da implementação em KNIME consistiu na limpeza e validação dos conjuntos de dados *Collisions* e *Vehicles*. O principal objetivo foi garantir que apenas os registos válidos e relevantes prosseguiam para as etapas de processamento e integração.

Para esse efeito, foram criados dois workflows independentes, destinados ao tratamento específico de cada conjunto de dados.

Limpeza e validação de Collisions

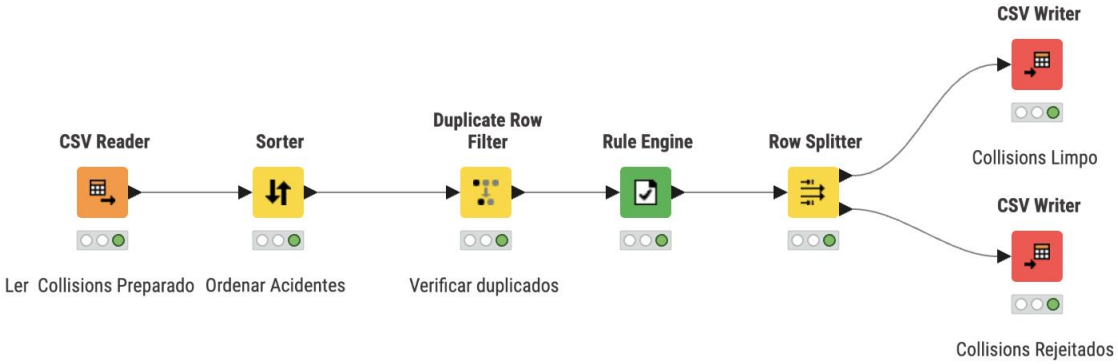


Figura 20- Workflow de limpeza e validação de Collisions no KNIME

A Figura 20 representa o workflow responsável pela limpeza dos dados relativos aos acidentes rodoviários. O processo inicia-se com a leitura do ficheiro Collisions_Preparado.csv, através do node CSV Reader. De seguida, os registos são ordenados pelo identificador collision_index, recorrendo ao node Sorter, de forma a preparar os dados para a identificação de duplicados.

Posteriormente, o node Duplicate Row Filter permite remover ocorrências repetidas com o mesmo identificador, garantindo que cada acidente é representado por um único registo.

A validação dos dados é realizada através de regras condicionais, utilizando nodes como Rule Engine e Row Splitter. Estas operações permitem verificar diferentes atributos, incluindo os identificadores, a localização geográfica e os valores associados às características dos acidentes.

Foi também criado o atributo auxiliar motivo_rejeicao, utilizado para identificar a razão pela qual determinados registos não cumprem as condições estabelecidas.

Os registos são separados em dois conjuntos: os que cumprem os critérios de validação e os que apresentam valores inválidos ou inconsistentes.

Por fim, os resultados são exportados para ficheiros CSV distintos, permitindo conservar os registos válidos para as transformações seguintes e manter os registos rejeitados para efeitos de análise e verificação.

Esta estratégia possibilita não apenas a limpeza dos dados, mas também uma maior rastreabilidade das decisões tomadas durante a validação.

Limpeza e validação de Vehicles

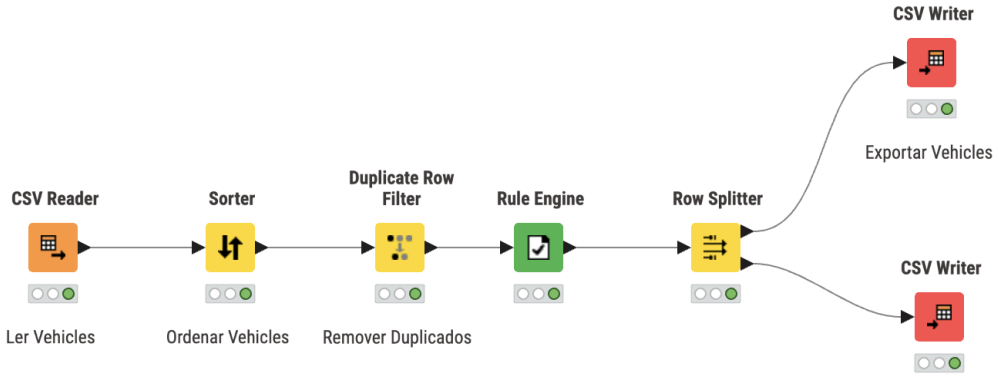


Figura 21- Workflow de limpeza e validação de Vehicles no KNIME

A Figura 21 apresenta o workflow destinado ao tratamento dos dados relativos aos veículos envolvidos nos acidentes. O processo começa com a leitura do ficheiro Vehicles_Preparado.csv, utilizando o node CSV Reader.

De seguida, os registos são ordenados através do node Sorter**,** considerando os campos collision_index e vehicle_reference. A combinação destes atributos permite identificar cada veículo associado a uma determinada colisão. Posteriormente, o node Duplicate Row Filt**er** é utilizado para remover registos duplicados com base nessa combinação de campos.

A validação dos restantes atributos é realizada através de regras condicionais, permitindo identificar valores inválidos ou inconsistentes nos dados relativos aos veículos e aos respetivos condutores.

Tal como no workflow anterior, os registos são classificados de acordo com os critérios definidos e separados entre válidos e rejeitados.

Por fim, os dados considerados válidos são exportados para um ficheiro CSV, que servirá de entrada ao workflow de processamento de veículos.

A utilização de processos de limpeza independentes permitiu aplicar validações adequadas à estrutura de cada conjunto de dados, garantindo maior consistência nas etapas seguintes.

4.2.2 - Processamento dos Dados

Após a limpeza e validação dos conjuntos de dados, foram desenvolvidos workflows destinados à transformação dos atributos originais em informação mais estruturada e fácil de interpretar.

Nesta etapa, destacam-se as operações de classificação de variáveis, conversão de códigos numéricos em descrições textuais, cálculo de novos atributos, agrupamentos e cruzamento de dados.

Processamento do dataset Vehicles

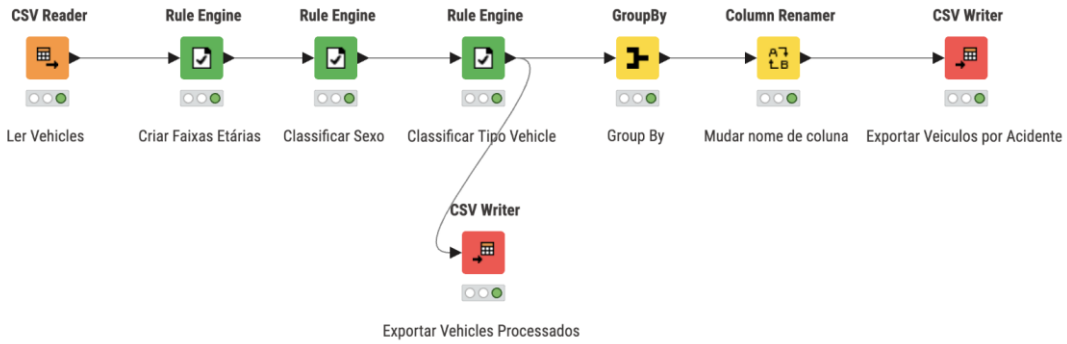


Figura 22- Workflow de processamento de Vehicles no KNIME

A Figura 22 representa o workflow responsável pelo processamento dos dados dos veículos e dos respetivos condutores. A transformação inicia-se com a leitura dos dados previamente validados. De seguida, são criados novos atributos destinados a facilitar a análise estatística dos condutores e dos veículos envolvidos.

Entre as operações realizadas, destaca-se a criação da variável faixa_etaria, que permite agrupar os condutores em categorias de idade, como menores de 18 anos, 18–24 anos, 25–34 anos, 35–54 anos, 55–64 anos e 65 ou mais anos, contemplando também situações em que a idade é desconhecida.

Foi igualmente criada a variável sexo_condutor, que converte os códigos existentes no dataset original em descrições textuais, permitindo distinguir os diferentes valores associados ao sexo dos condutores.

Outra transformação consistiu na criação do atributo categoria_veiculo, que permite interpretar os códigos relativos aos tipos de veículos através de categorias descritivas.

Estas classificações foram implementadas através de operações de transformação e regras condicionais, recorrendo a nodes de manipulação e mapeamento de atributos.

Adicionalmente, foi utilizado o node GroupBy para agrupar os registos pelo campo collision_index e calcular o número de veículos associados a cada acidente.

Como resultado, foram produzidos dois ficheiros CSV:

- Vehicles_Processado.csv, contendo os dados dos veículos enriquecidos com os novos atributos;
- Veiculos_Por_Acidente.csv, contendo o número de veículos envolvidos em cada acidente.

O segundo ficheiro é posteriormente utilizado no processamento dos acidentes, permitindo associar a cada ocorrência o respetivo número de veículos.

Processamento do dataset Collisions

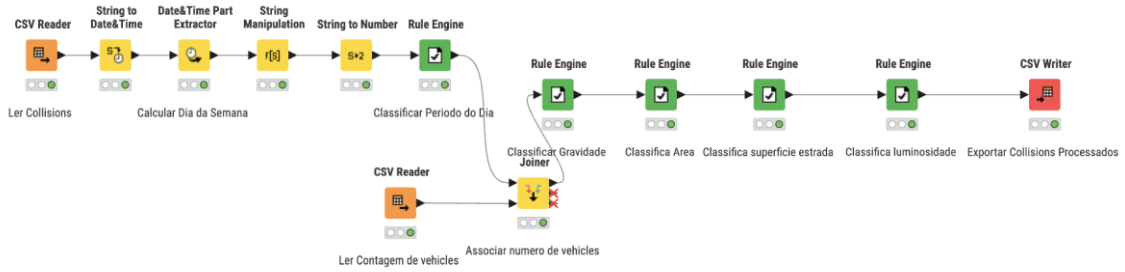


Figura 23-Workflow de processamento de Collisions no KNIME

A Figura 23 apresenta o workflow responsável por transformar os dados relativos aos acidentes rodoviários. O processo inicia-se com a leitura do conjunto de dados *Collisions* previamente limpo e validado.

Uma das principais operações consiste no tratamento dos atributos temporais. A partir dos campos date e time, foram criadas variáveis que permitem caracterizar os acidentes segundo o dia da semana e o período do dia.

A variável dia_semana identifica o dia em que ocorreu cada acidente, enquanto hora permite obter a respetiva hora. Com base nesta informação, foi criada a variável periodo_dia, que classifica as ocorrências em quatro períodos: madrugada, manhã, tarde e noite.

Adicionalmente, foi realizado o cruzamento com os resultados do processamento dos veículos, através do node Joiner, utilizando o campo collision_index como chave de associação.

Esta operação permitiu acrescentar o atributo numero_veiculos aos registos dos acidentes.

Foram também realizadas operações de conversão de códigos numéricos em descrições textuais, permitindo interpretar atributos relacionados com a gravidade

dos acidentes, o tipo de zona, o estado da superfície da estrada e as condições de luminosidade.

Desta forma, foram criados atributos como descricao_gravidade, descricao_tipo_area, descricao_superficie_estrada e descricao_luminosidade.

Por fim, os dados processados foram exportados para o ficheiro Collisions_Processado_KNIME.csv, que constitui uma das principais entradas do workflow de integração com os dados meteorológicos.

4.2.3 - Preparação dos Dados Meteorológicos

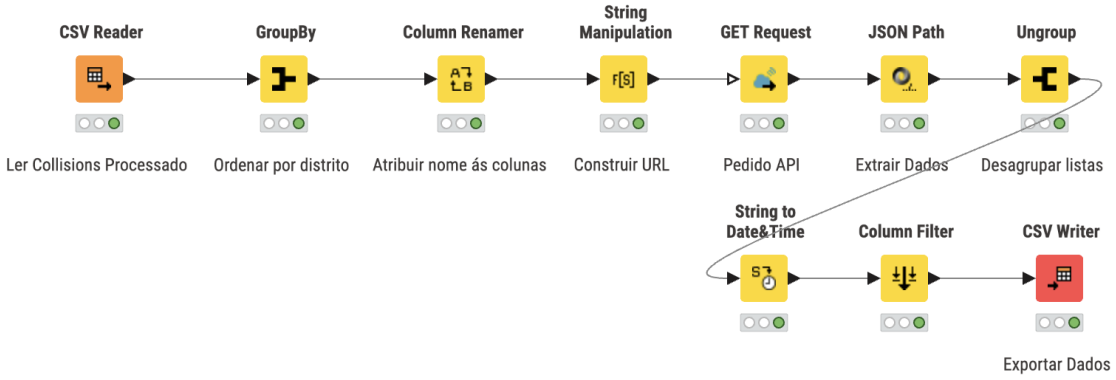


Figura 24 - Workflow de preparação dos dados meteorológicos no KNIME

A Figura 24 representa o workflow desenvolvido com o objetivo de obter e preparar informação meteorológica histórica para complementar os dados dos acidentes rodoviários.

Para a recolha dos dados foi utilizada a API histórica Open-Meteo, através de pedidos HTTP realizados pelo node GET Request. À semelhança da estratégia descrita anteriormente, optou-se por obter os dados meteorológicos por distrito de Londres, em vez de realizar um pedido individual para cada acidente. Esta abordagem permitiu reduzir o número de chamadas à API e tornar o processo de recolha mais eficiente.

Inicialmente, os acidentes foram agrupados segundo o atributo local_authority_ons_district, correspondente à identificação administrativa das zonas analisadas.

Para validar os códigos dos distritos, foi utilizada uma expressão regular (*Regex*) correspondente ao padrão ^E09[0-9]{6}$.

Posteriormente, foram calculadas coordenadas geográficas representativas das diferentes zonas, permitindo construir os endereços utilizados nos pedidos à API.

Os pedidos efetuados através do node GET Request devolveram informação meteorológica histórica em formato JSON.

A extração dos dados foi realizada através do node JSON Path, permitindo obter valores horários relativos à temperatura, precipitação, velocidade do vento e condições meteorológicas.

Uma vez que as respostas da API continham conjuntos de valores organizados em estruturas JSON, foi necessário proceder à sua transformação em registos tabulares. Para esse efeito, foram utilizados nodes como Ungroup**,** permitindo associar cada observação meteorológica à respetiva data, hora e localização.

Os resultados foram posteriormente organizados e exportados para o ficheiro Meteorologia_Londres.csv.

Este ficheiro constitui a fonte meteorológica utilizada na etapa seguinte de integração dos dados.

4.2.4 - Integração dos Dados

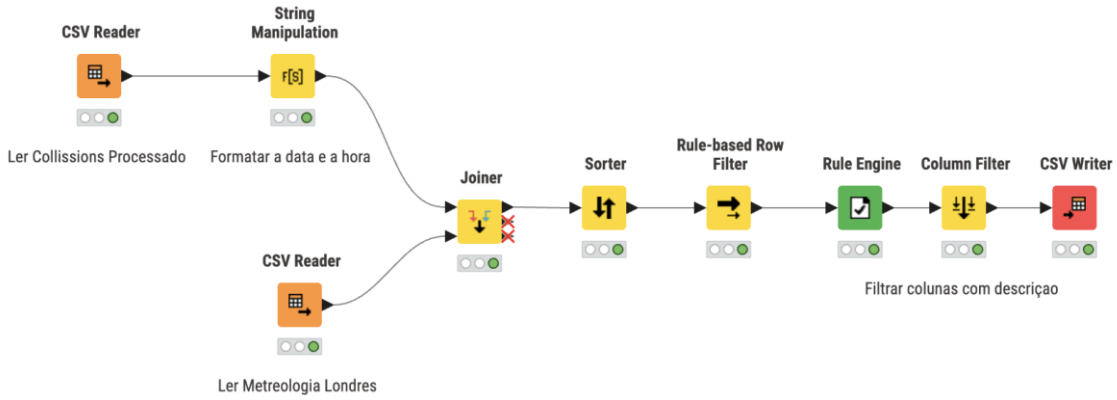


Figura 25 - Workflow de integração dos acidentes com os dados meteorológicos no KNIME

A Figura 25 representa o workflow responsável por relacionar os acidentes rodoviários previamente processados com a informação meteorológica obtida através da API Open-Meteo.

O processo inicia-se com a leitura dos ficheiros Collisions_Processado.csv e Meteorologia_Londres.csv.

Uma vez que os dados meteorológicos foram recolhidos com periodicidade horária, foi necessário preparar os atributos temporais dos acidentes para permitir uma correspondência adequada.

Para estabelecer a relação entre os conjuntos de dados, foram considerados dois critérios principais: o distrito onde ocorreu o acidente e a respetiva referência temporal.

A integração foi realizada através do node Joiner, permitindo associar aos registos dos acidentes os valores meteorológicos correspondentes.

Após a operação de junção, foram efetuadas validações adicionais para identificar situações em que não foi possível estabelecer uma correspondência meteorológica.

Para esse efeito, foi utilizado um Rule-based Row Filter, permitindo selecionar os registos com informação meteorológica válida.

O conjunto de dados final passou, assim, a incluir simultaneamente atributos relativos aos acidentes, às características rodoviárias, ao número de veículos envolvidos e às condições meteorológicas associadas às ocorrências.

Por fim, o resultado foi exportado para o ficheiro Collisions_Com_Meteorologia.csv.

Esta etapa constitui um dos principais objetivos do projeto, uma vez que permite integrar informação proveniente de fontes distintas e disponibilizar dados enriquecidos para armazenamento e análise.

4.2.5 - Carregamento dos Dados no PostgreSQL

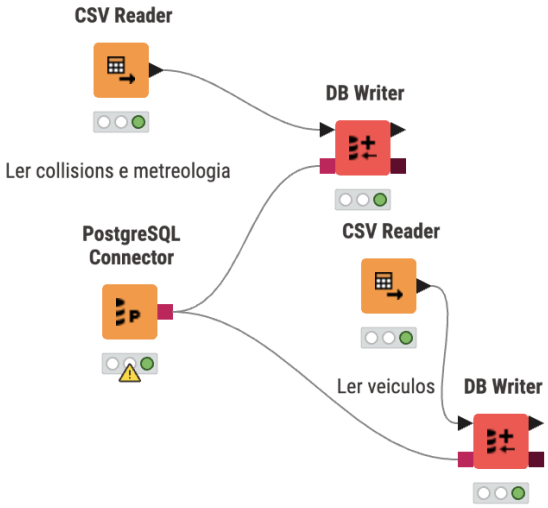


Figura 26 - Workflow de carregamento dos dados no PostgreSQL através do KNIME

A Figura 26 apresenta o workflow desenvolvido com o objetivo de armazenar os dados processados numa base de dados relacional.

Para esta etapa foi utilizado o PostgreSQL, através de uma ligação estabelecida pelo node PostgreSQL Connector.

A base de dados, denominada acidentes_londres, foi organizada em duas tabelas principais: collisions e vehicles.

A tabela collisions contém os registos dos acidentes, incluindo os atributos resultantes do processamento e da integração meteorológica. Por sua vez, a tabela vehicles armazena as características dos veículos e dos condutores associados a cada ocorrência.

As tabelas estão relacionadas através do campo collision_index, que funciona como identificador único dos acidentes.

Para garantir a integridade dos dados, foram definidas chaves primárias nas duas tabelas e uma chave estrangeira na tabela vehicles, referenciando a tabela collisions.

O carregamento foi implementado através de dois ramos independentes, cada um constituído por um node CSV Reader e um node DB Writer, ligados ao mesmo PostgreSQL Connector.

O primeiro ramo é responsável pela inserção dos acidentes na tabela collisions, enquanto o segundo procede ao carregamento dos veículos na tabela vehicles.

Durante a configuração dos nodes DB Writer, foi realizada a correspondência entre os atributos dos ficheiros de entrada e as colunas das tabelas PostgreSQL. As colunas auxiliares de validação e os atributos redundantes não necessários ao modelo final foram excluídos da operação de escrita.

O carregamento foi efetuado respeitando a relação entre as tabelas, inserindo primeiro os acidentes e posteriormente os veículos.

Resultados do carregamento

```
acidentes_londres=#
acidentes_londres=# SELECT 'collisions' AS tabela, COUNT(*) AS total_registos
acidentes_londres-# FROM public.collisions
acidentes_londres-#
acidentes_londres-# UNION ALL
acidentes_londres-#
acidentes_londres-# SELECT 'vehicles' AS tabela, COUNT(*) AS total_registos
acidentes_londres-# FROM public.vehicles;
   tabela   | total_registos
------------+----------------
 collisions |           1701
 vehicles   |           3120
(2 rows)
```

Figura 27 - Verificação dos registos armazenados no PostgreSQL

Após a execução do workflow, foram armazenados 1 701 registos de acidentes na tabela collisions e 3 120 registos de veículos na tabela vehicles.

A correta inserção dos dados foi confirmada através de consultas SQL, incluindo operações de contagem e junção entre as duas tabelas.

Foram igualmente verificadas as restrições de integridade referencial, garantindo que os registos dos veículos estão associados a acidentes existentes.

Esta etapa permitiu consolidar os dados transformados numa estrutura relacional, preparada para a execução de consultas estatísticas e posterior exportação dos resultados.

4.2.6 - Exportação dos Resultados e Envio de Notificações

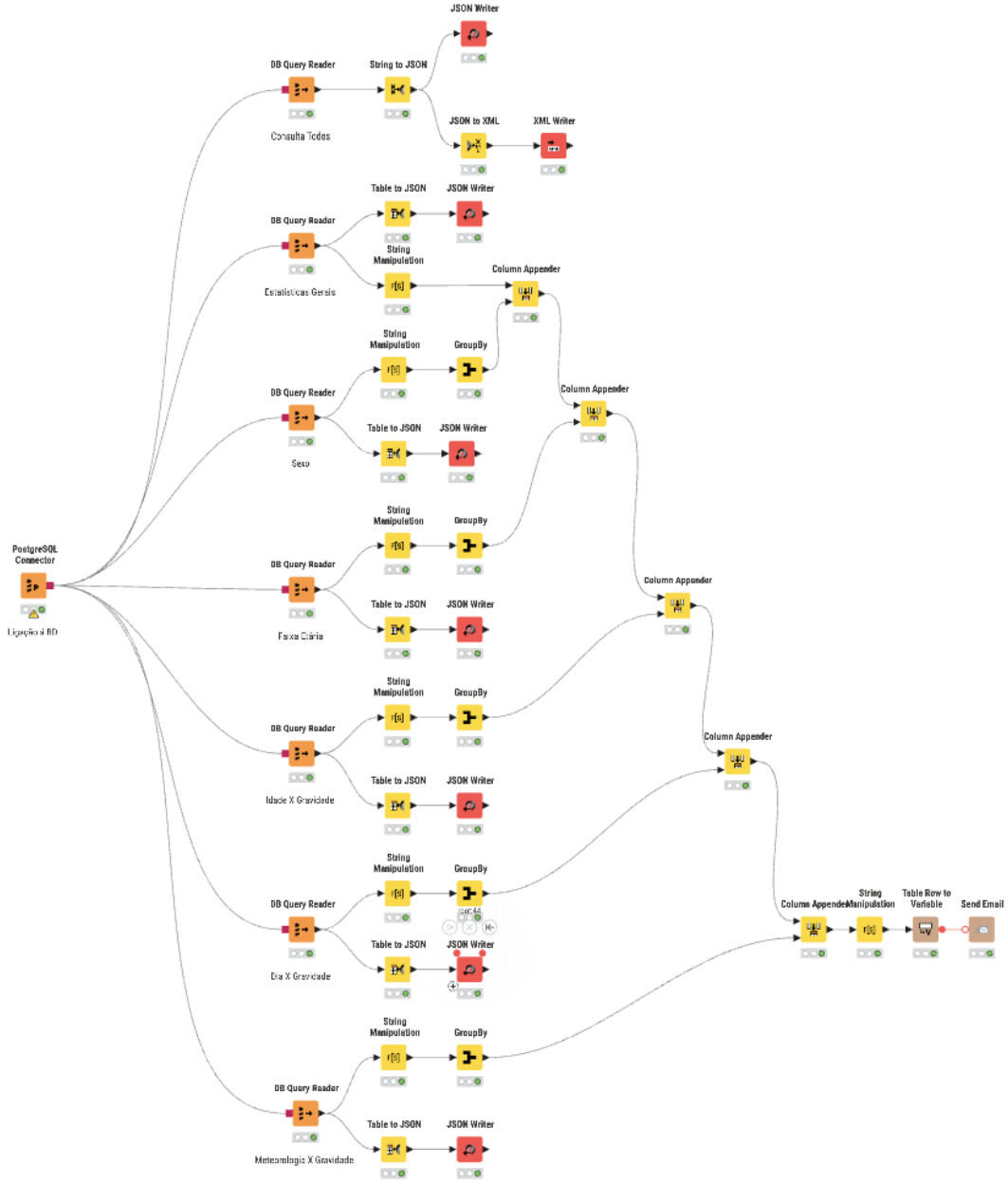


Figura 28 - — Workflow de exportação de resultados e envio de email no KNIME

A Figura 28 representa o workflow responsável pela consulta dos dados armazenados no PostgreSQL, produção de estatísticas, exportação de ficheiros e envio automático de um relatório por correio eletrónico.

O processo inicia-se com o estabelecimento da ligação à base de dados acidentes_londres, através do node PostgreSQL Connector.

De seguida, são utilizados vários nodes DB Query Reader, responsáveis pela execução de consultas SQL sobre as tabelas collisions e vehicles.

Uma das principais funcionalidades deste workflow consiste na geração de um documento JSON integrado, contendo os dados dos acidentes e os respetivos veículos associados.

Para esse efeito, foi utilizada uma consulta SQL que relaciona as duas tabelas e constrói uma estrutura JSON hierárquica. Posteriormente, os dados foram convertidos através do node String to JSON e exportados utilizando o node JSON Writer.

Foi também implementada a conversão da estrutura JSON para XML, recorrendo ao node JSON to XML e ao XML Writer.

Desta forma, foram produzidos ficheiros integrados em dois formatos distintos, permitindo demonstrar a conversão e disponibilização de informação estruturada.

Produção de estatísticas

Adicionalmente, foram desenvolvidas seis consultas estatísticas, com o objetivo de caracterizar os acidentes e os veículos envolvidos.

As análises realizadas incluem:

- Estatísticas gerais dos acidentes, vítimas, veículos e temperatura média;
- Distribuição dos condutores por sexo;
- Distribuição dos condutores por faixa etária;
- Relação entre a faixa etária dos condutores e a gravidade dos acidentes;
- Relação entre o período do dia e a gravidade dos acidentes;
- Relação entre as condições meteorológicas e a gravidade dos acidentes.

Os resultados destas consultas foram transformados em estruturas JSON através do node Table to JSON, sendo posteriormente exportados pelo JSON Writer.

Foram assim gerados seis ficheiros estatísticos, destinados à utilização num dashboard desenvolvido em Node-RED.

A realização destas consultas permitiu transformar os dados armazenados no PostgreSQL em indicadores estatísticos úteis para a interpretação dos resultados do projeto.

Estatísticas Gerais

```
SELECT
    (SELECT COUNT(*)
     FROM public.collisions) AS total_acidentes,

    (SELECT SUM(number_of_casualties)
     FROM public.collisions) AS total_vitimas,

    (SELECT COUNT(*)
     FROM public.vehicles) AS total_veiculos,

    (SELECT ROUND(AVG(temperatura)::numeric, 2)
     FROM public.collisions) AS temperatura_media,

    (SELECT COUNT(*)
     FROM public.collisions
     WHERE descricao_gravidade = 'Grave') AS total_graves,

    (SELECT COUNT(*)
     FROM public.collisions
     WHERE descricao_gravidade = 'Fatal') AS total_fatais;
```

total_acidentes bigint	total_vitimas bigint	total_veiculos bigint	temperatura_media numeric	total_graves bigint	total_fatais bigint
1701	1965	3120	20.32	340	7

Estatísticas por sexo

```
SELECT
    sexo_condutor,
    COUNT(*) AS total_condutores
FROM public.vehicles
GROUP BY sexo_condutor
ORDER BY total_condutores DESC;
```

sexo_condutor character varying	total_condutores bigint
Masculino	1891
Não Identificado	644
Feminino	483
Desconhecido	102

Estatística por Faixa Etária

```
SELECT
    faixa_etaria,
    COUNT(*) AS total_condutores
FROM public.vehicles
GROUP BY faixa_etaria
ORDER BY total_condutores DESC;
```

faixa_etaria character varying	total_condutores bigint
35-54	878
Desconhecida	709
25-34	704
18-24	360
55-64	274
65 ou mais	148
Menor de 18	47

Estatística Faixa Etária x Gravidade

```sql
SELECT
    v.faixa_etaria,
    c.descricao_gravidade,
    COUNT(*) AS total_veiculos
FROM public.vehicles v
INNER JOIN public.collisions c
    ON v.collision_index = c.collision_index
GROUP BY
    v.faixa_etaria,
    c.descricao_gravidade
ORDER BY
    v.faixa_etaria,
    c.descricao_gravidade;
```

faixa_etaria character varying	descricao_gravidade character varying	total_veiculos bigint
18-24	Fatal	1
18-24	Grave	70
18-24	Ligeiro	289
25-34	Fatal	2
25-34	Grave	146
25-34	Ligeiro	556
35-54	Fatal	5
35-54	Grave	169
35-54	Ligeiro	704
55-64	Fatal	3
55-64	Grave	64
55-64	Ligeiro	207
65 ou mais	Grave	30
65 ou mais	Ligeiro	118
Desconhecida	Fatal	4
Desconhecida	Grave	113
Desconhecida	Ligeiro	592
Menor de 18	Grave	16
Menor de 18	Ligeiro	31

Estatística Período do Dia x Gravidade

```sql
SELECT
    periodo_dia,
    descricao_gravidade,
    COUNT(*) AS total_acidentes
FROM public.collisions
GROUP BY
    periodo_dia,
    descricao_gravidade
ORDER BY
    CASE periodo_dia
        WHEN 'Madrugada' THEN 1
        WHEN 'Manhã' THEN 2
        WHEN 'Tarde' THEN 3
        WHEN 'Noite' THEN 4
        ELSE 5
    END,
    descricao_gravidade;
```

periodo_dia character varying	descricao_gravidade character varying	total_acidentes bigint
Madrugada	Grave	42
Madrugada	Ligeiro	93
Manhã	Fatal	1
Manhã	Grave	65
Manhã	Ligeiro	297
Tarde	Fatal	5
Tarde	Grave	126
Tarde	Ligeiro	536
Noite	Fatal	1
Noite	Grave	107
Noite	Ligeiro	428

Estatística Meteorologia x Gravidade

```
SELECT
    condicao_meteorologica,
    descricao_gravidade,
    COUNT(*) AS total_acidentes
FROM public.collisions
GROUP BY
    condicao_meteorologica,
    descricao_gravidade
ORDER BY
    condicao_meteorologica,
    descricao_gravidade;
```

condicao_meteorologica character varying	descricao_gravidade character varying	total_acidentes bigint
Céu limpo	Fatal	1
Céu limpo	Grave	85
Céu limpo	Ligeiro	331
Chuva fraca	Grave	1
Chuva fraca	Ligeiro	4
Chuva moderada	Grave	3
Chuva moderada	Ligeiro	5
Chuvisco fraco	Fatal	1
Chuvisco fraco	Grave	30
Chuvisco fraco	Ligeiro	155
Chuvisco intenso	Ligeiro	4
Chuvisco moderado	Grave	4
Chuvisco moderado	Ligeiro	15
Encoberto	Fatal	3
Encoberto	Grave	146
Encoberto	Ligeiro	599
Maioritariamente limpo	Fatal	1
Maioritariamente limpo	Grave	42
Maioritariamente limpo	Ligeiro	139
Parcialmente nublado	Fatal	1
Parcialmente nublado	Grave	29
Parcialmente nublado	Ligeiro	102

Envio automático do relatório por email

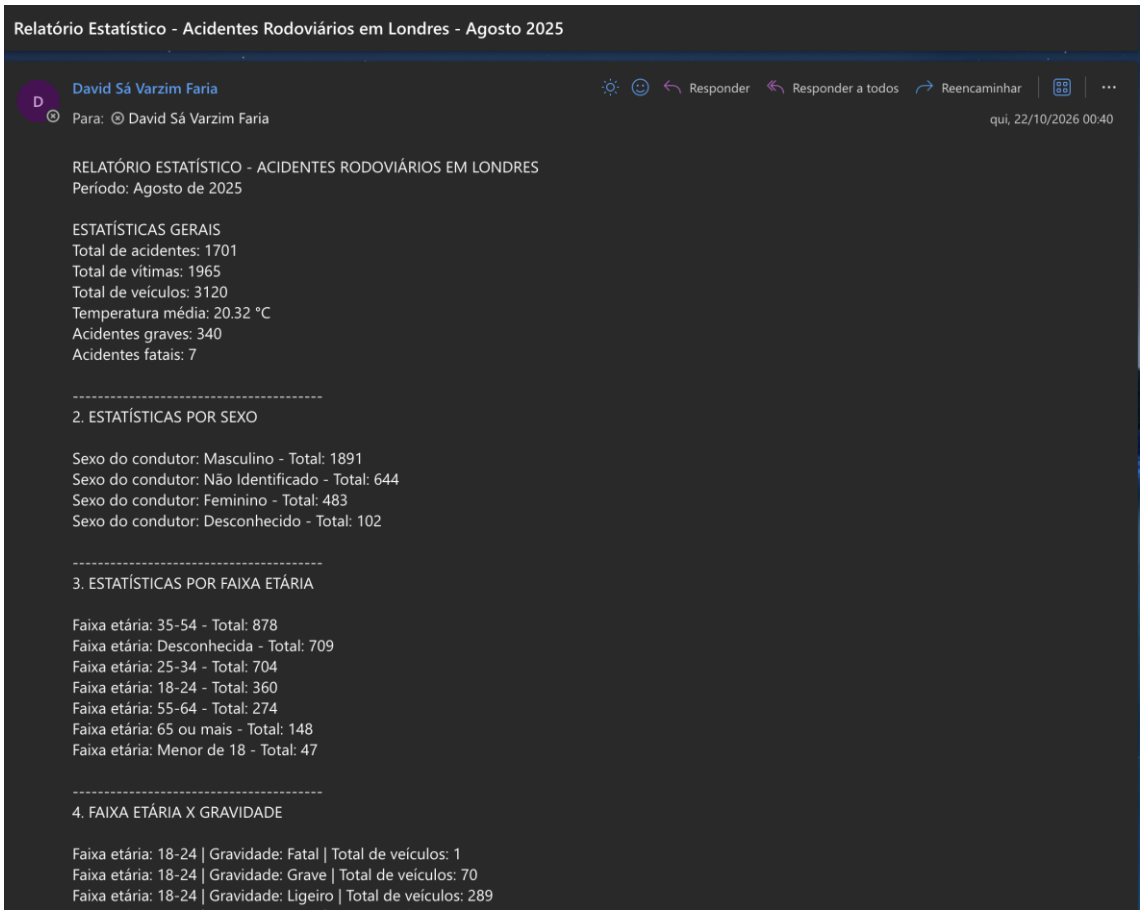


Figura 29 - Execução do envio do relatório estatístico por email

Outra funcionalidade desenvolvida neste workflow consistiu no envio automático de um relatório estatístico através de correio eletrónico.

Para esse efeito, os resultados das diferentes consultas foram convertidos em texto, recorrendo a operações de manipulação e agregação de dados.

Posteriormente, as informações foram reunidas numa única estrutura, contendo os principais indicadores estatísticos relativos aos acidentes rodoviários em Londres durante agosto de 2025.

Foi utilizado o node Table Row to Variable para disponibilizar o conteúdo do relatório como variável de fluxo (*Flow Variable*), permitindo integrar dinamicamente os resultados no corpo da mensagem.

O envio foi realizado através do node Send Email, utilizando uma ligação SMTP previamente configurada.

A funcionalidade foi testada através do envio e receção do relatório, confirmando a disponibilização automática dos resultados por correio eletrónico.

Esta operação constitui uma componente adicional de integração com serviços externos, demonstrando a capacidade do KNIME para comunicar os resultados do processo ETL a outros destinatários.

4.2.7 - Monitorização dos Logs de Ligação ao PostgreSQL

Com o objetivo de complementar o processo de integração de dados e permitir um maior acompanhamento das operações de ligação à base de dados, foi desenvolvido um workflow específico para a monitorização dos logs associados ao PostgreSQL Connector, utilizado na implementação em KNIME.

Esta funcionalidade permite consultar e organizar as mensagens geradas automaticamente pelo KNIME Analytics Platform durante a utilização do componente responsável pela ligação à base de dados PostgreSQL.

Para esse efeito, foi utilizado o ficheiro knime.log, que contém mensagens produzidas pela plataforma, incluindo informações sobre execuções concluídas, avisos e erros.

Através do tratamento deste ficheiro, pretendeu-se disponibilizar um registo estruturado dos acontecimentos associados ao node PostgreSQL Connector, facilitando a sua consulta e análise.

Leitura e tratamento dos logs

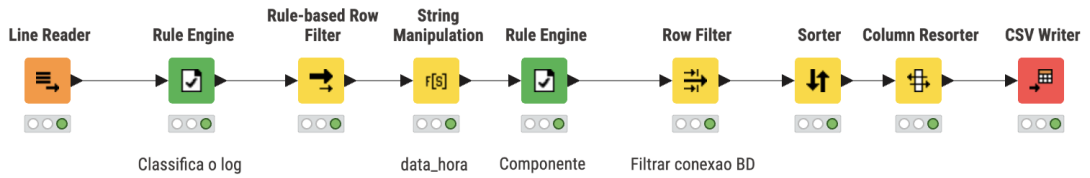


Figura 30 - Workflow de monitorização dos logs de ligação ao PostgreSQL no KNIME

A Figura X representa o workflow responsável pela leitura, classificação, filtragem e exportação das mensagens relacionadas com o PostgreSQL Connector.

O processo inicia-se com o node Line Reader, que permite importar o conteúdo do ficheiro knime.log, considerando cada linha como um registo individual.

De seguida, foi utilizado o node Rule Engine para classificar as mensagens em diferentes categorias, nomeadamente EXECUÇÃO TERMINADA, AVISO e ERRO, permitindo distinguir os acontecimentos registados pela plataforma.

Posteriormente, foi aplicado o node Rule-based Row Filter, com o objetivo de remover mensagens não relevantes para a análise, reduzindo o volume de registos e mantendo apenas as categorias de eventos pretendidas.

Através do node String Manipulation, foi criado o atributo data_hora, extraindo a data e a hora presentes no início das mensagens do ficheiro de logs. Foi utilizado um segundo Rule Engine para identificar o componente relacionado com cada mensagem, criando o atributo componente.

De seguida, através do node Row Filter, foram selecionados exclusivamente os registos associados ao PostgreSQL Connector, excluindo mensagens relativas aos restantes componentes do KNIME. Os registos selecionados foram posteriormente ordenados através do node Sorter, utilizando o atributo data_hora em ordem decrescente, permitindo apresentar os acontecimentos mais recentes no início da tabela. O node Column Resorter foi utilizado para reorganizar os atributos, colocando a data e hora, o componente e o tipo de evento antes da mensagem original.

Por fim, os resultados foram exportados para um ficheiro CSV através do node CSV Writer, permitindo conservar os registos de monitorização para posterior consulta.

Resultados da monitorização

Logs_PostgreSQL

data_hora	componente	tipo_evento	mensagem_log
2026-10-22 00:40:53,063	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-22 00:40:53,063 : INFO : KNIME-Worker-87-PostgreSQL Connector 6:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 6:1 : PostgreSQL Connector 6:1 End execute (0 secs)
2026-10-22 00:40:50,230	PostgreSQL Connector	AVISO	2026-10-22 00:40:50,230 : WARN : ModalContext : : Node : PostgreSQL Connector : 6:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-22 00:16:05,823	PostgreSQL Connector	AVISO	2026-10-22 00:16:05,823 : WARN : ModalContext : : Node : PostgreSQL Connector : 7:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-22 00:13:37,551	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-22 00:13:37,551 : INFO : KNIME-Worker-50-PostgreSQL Connector 6:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 6:1 : PostgreSQL Connector 6:1 End execute (0 secs)
2026-10-22 00:13:34,070	PostgreSQL Connector	AVISO	2026-10-22 00:13:34,070 : WARN : ModalContext : : Node : PostgreSQL Connector : 6:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:48:56,905	PostgreSQL Connector	AVISO	2026-10-21 23:48:56,905 : WARN : ModalContext : : Node : PostgreSQL Connector : 6:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:48:33,104	PostgreSQL Connector	AVISO	2026-10-21 23:48:33,104 : WARN : ModalContext : : Node : PostgreSQL Connector : 4:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:44:39,457	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:44:39,457 : INFO : KNIME-Worker-19-PostgreSQL Connector 4:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 4:1 : PostgreSQL Connector 4:1 End execute (0 secs)
2026-10-21 23:32:50,751	PostgreSQL Connector	AVISO	2026-10-21 23:32:50,751 : WARN : ModalContext : : Node : PostgreSQL Connector : 5:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:30:11,055	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:30:11,055 : INFO : KNIME-Worker-15-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:29:55,280	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:29:55,280 : INFO : KNIME-Worker-3-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:29:43,541	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:29:43,541 : INFO : KNIME-Worker-2-PostgreSQL Connector 4:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 4:1 : PostgreSQL Connector 4:1 End execute (0 secs)
2026-10-21 23:29:40,583	PostgreSQL Connector	AVISO	2026-10-21 23:29:40,583 : WARN : ModalContext : : Node : PostgreSQL Connector : 4:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:29:17,262	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:29:17,262 : INFO : KNIME-Worker-3-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:26:42,166	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:26:42,166 : INFO : KNIME-Worker-1-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:25:31,327	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:25:31,327 : INFO : KNIME-Worker-0-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:22:19,674	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:22:19,674 : INFO : KNIME-Worker-0-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 23:21:46,906	PostgreSQL Connector	AVISO	2026-10-21 23:21:46,906 : WARN : comm-pool-thread-1 : : Node : PostgreSQL Connector : 3:1 : Database host is not defined.
2026-10-21 23:15:28,909	PostgreSQL Connector	AVISO	2026-10-21 23:15:28,909 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 23:13:56,911	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 23:13:56,911 : INFO : KNIME-Worker-73-PostgreSQL Connector 4:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 4:1 : PostgreSQL Connector 4:1 End execute (0 secs)
2026-10-21 23:13:20,569	PostgreSQL Connector	AVISO	2026-10-21 23:13:20,569 : WARN : ModalContext : : Node : PostgreSQL Connector : 4:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:58:49,081	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 22:58:49,081 : INFO : KNIME-Worker-40-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 22:55:11,548	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 22:55:11,548 : INFO : KNIME-Worker-23-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 22:45:00,762	PostgreSQL Connector	AVISO	2026-10-21 22:45:00,762 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:44:00,800	PostgreSQL Connector	AVISO	2026-10-21 22:44:00,800 : WARN : ModalContext : : Node : PostgreSQL Connector : 5:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:43:14,589	PostgreSQL Connector	AVISO	2026-10-21 22:43:14,589 : WARN : ModalContext : : Node : PostgreSQL Connector : 5:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:41:27,117	PostgreSQL Connector	AVISO	2026-10-21 22:41:27,117 : WARN : ModalContext : : Node : PostgreSQL Connector : 4:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:37:09,237	PostgreSQL Connector	AVISO	2026-10-21 22:37:09,237 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:32:37,661	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 22:32:37,661 : INFO : KNIME-Worker-1-PostgreSQL Connector 4:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 4:1 : PostgreSQL Connector 4:1 End execute (0 secs)
2026-10-21 22:31:52,779	PostgreSQL Connector	AVISO	2026-10-21 22:31:52,779 : WARN : ModalContext : : Node : PostgreSQL Connector : 4:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 22:31:48,654	PostgreSQL Connector	AVISO	2026-10-21 22:31:48,654 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 20:57:37,506	PostgreSQL Connector	AVISO	2026-10-21 20:57:37,506 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 15:56:29,210	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 15:56:29,210 : INFO : KNIME-Worker-0-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 15:56:14,236	PostgreSQL Connector	AVISO	2026-10-21 15:56:14,236 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 11:21:12,255	PostgreSQL Connector	AVISO	2026-10-21 11:21:12,255 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 11:17:40,997	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 11:17:40,997 : INFO : KNIME-Worker-0-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 11:17:28,330	PostgreSQL Connector	ERRO	2026-10-21 11:17:28,330 : ERROR : KNIME-Worker-0-PostgreSQL Connector 3:1 : : Node : PostgreSQL Connector : 3:1 : Execute failed: FATAL: database "acidentes_londres" does not exist
2026-10-21 11:16:56,670	PostgreSQL Connector	AVISO	2026-10-21 11:16:56,670 : WARN : comm-pool-thread-2 : : Node : PostgreSQL Connector : 3:1 : Database host is not defined.
2026-10-21 11:16:26,010	PostgreSQL Connector	AVISO	2026-10-21 11:16:26,010 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 10:24:56,296	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-21 10:24:56,296 : INFO : KNIME-Worker-1-PostgreSQL Connector 3:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 3:1 : PostgreSQL Connector 3:1 End execute (0 secs)
2026-10-21 10:22:10,105	PostgreSQL Connector	AVISO	2026-10-21 10:22:10,105 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-21 09:52:27,280	PostgreSQL Connector	AVISO	2026-10-21 09:52:27,280 : WARN : ModalContext : : Node : PostgreSQL Connector : 3:1 : DB Connection no longer available. Go to advanced settings to enable connection restoration.
2026-10-20 15:03:10,583	PostgreSQL Connector	EXECUÇÃO TERMINADA	2026-10-20 15:03:10,583 : INFO : KNIME-Worker-303-PostgreSQL Connector 4:1 : : LocalNodeExecutionJob : PostgreSQL Connector : 4:1 : PostgreSQL Connector 4:1 End execute (0 secs)

Figura 31 - Registos de execução do PostgreSQL Connector organizados por data decrescente

O resultado apresenta uma tabela constituída pelos seguintes atributos:

- **data_hora:** data e hora em que o acontecimento foi registado;
- componente: identificação do node associado à mensagem;
- **tipo_evento:** classificação do acontecimento como execução terminada, aviso ou erro;
- **mensagem_log:** conteúdo original da mensagem gerada pelo KNIME.

Através deste processo, foi possível obter um histórico organizado dos eventos associados ao PostgreSQL Connector, permitindo identificar execuções concluídas e eventuais avisos ou erros relacionados com este componente.

A geração dos logs é automática, enquanto a respetiva leitura, transformação e exportação são realizadas quando o workflow de monitorização é executado.

Esta funcionalidade contribui para uma maior rastreabilidade das operações de ligação à base de dados, complementando as etapas de integração e persistência desenvolvidas no projeto.

4.2.8 - Visualização dos Resultados com Node-Red

Com o objetivo de facilitar a interpretação dos resultados obtidos através do processo ETL, foi desenvolvido um dashboard interativo utilizando a plataforma Node-RED, em conjunto com a extensão FlowFuse Dashboard 2.0.

Esta funcionalidade permite apresentar de forma gráfica e organizada as estatísticas relativas aos acidentes rodoviários ocorridos em Londres durante agosto de 2025, disponibilizando indicadores gerais e análises relacionadas com os condutores, a gravidade dos acidentes e as condições meteorológicas.

A utilização do Node-RED constitui uma componente complementar à implementação em KNIME, permitindo integrar os resultados provenientes da base de dados PostgreSQL numa interface visual destinada à análise e apresentação da informação.

Preparação e Integração dos Dados

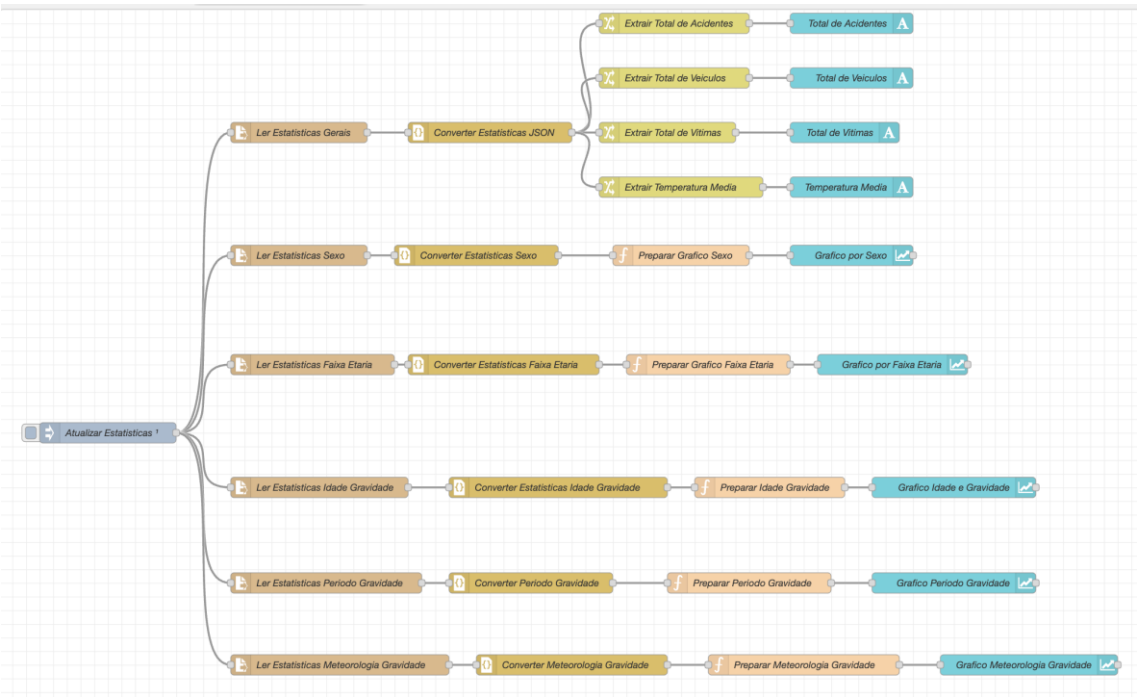


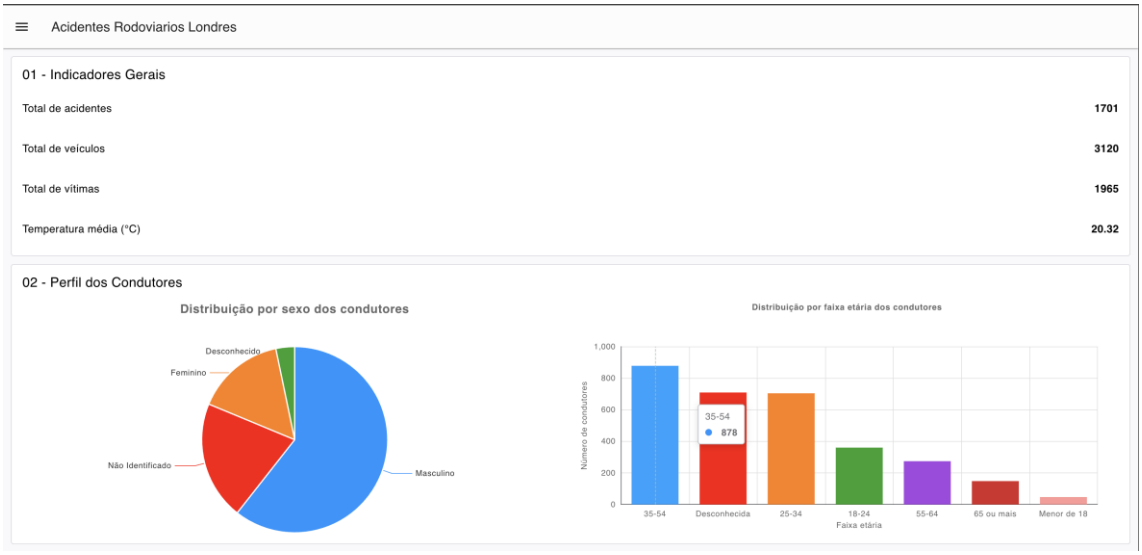
Figura 32 - Workflow Node-RED responsável pela leitura e preparação dos dados estatísticos

A Figura X apresenta o fluxo desenvolvido no Node-RED para a leitura, transformação e disponibilização dos dados estatísticos. Os dados utilizados no dashboard são provenientes dos ficheiros JSON gerados pelo workflow 08_Exportacao_Resultados do KNIME, através de consultas SQL executadas sobre a base de dados PostgreSQL. A comunicação entre o KNIME e o Node-RED é realizada através destes ficheiros JSON, permitindo separar o processamento dos dados da respetiva visualização.

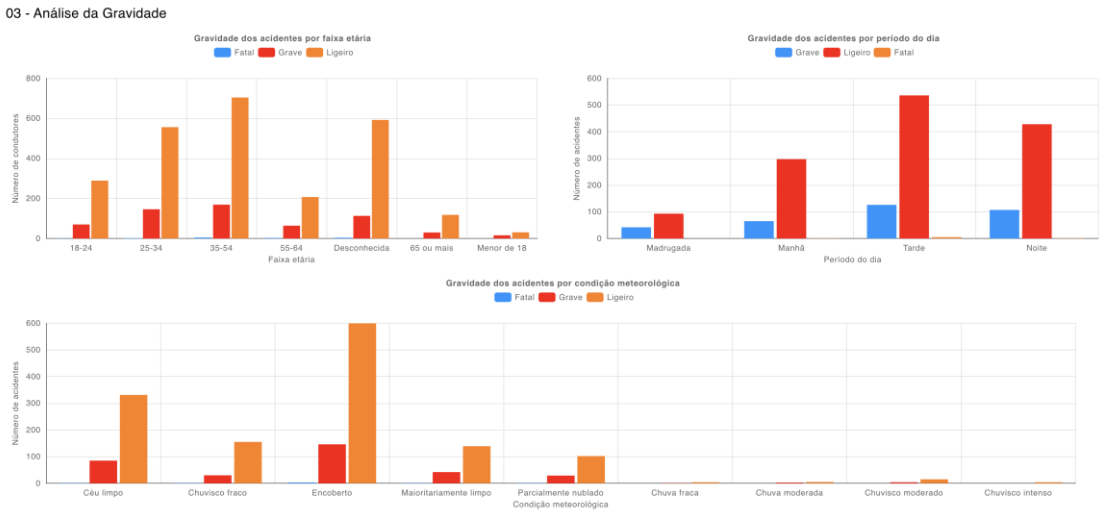
O fluxo inicia-se com nodes Inject, responsáveis por desencadear a atualização dos dados estatísticos. De seguida, os nodes Read File permitem ler os ficheiros JSON exportados pelo KNIME. Posteriormente, os nodes JSON convertem o conteúdo dos ficheiros em objetos JavaScript, permitindo o tratamento dos dados no Node-RED. Foram utilizados nodes Change para extrair os valores dos indicadores gerais e nodes Function para preparar os conjuntos de dados destinados à representação gráfica. Após estas transformações, os resultados são encaminhados para os componentes visuais do FlowFuse Dashboard, nomeadamente os nodes Text e Chart.

Desta forma, o Node-RED permite apresentar os resultados do processo ETL sem necessidade de consultar diretamente as tabelas da base de dados.

Organização do Dashboard



[Inserir Figura X — Vista geral do dashboard desenvolvido em Node-RED]



Estas análises permitem explorar possíveis padrões presentes nos dados. Contudo, as relações observadas representam associações estatísticas, não permitindo estabelecer, por si só, relações de causalidade entre os fatores analisados e a ocorrência ou gravidade dos acidentes.

Atualização dos Dados

A atualização do dashboard é realizada através da execução dos nodes Inject, que desencadeiam uma nova leitura dos ficheiros JSON gerados pelo KNIME.

Sempre que os ficheiros estatísticos são atualizados e os respetivos fluxos do Node-RED são novamente executados, os componentes visuais passam a apresentar os resultados disponibilizados.

Assim, o dashboard não utiliza um fluxo contínuo de dados em tempo real, mas sim um mecanismo de atualização baseado na leitura dos ficheiros exportados.

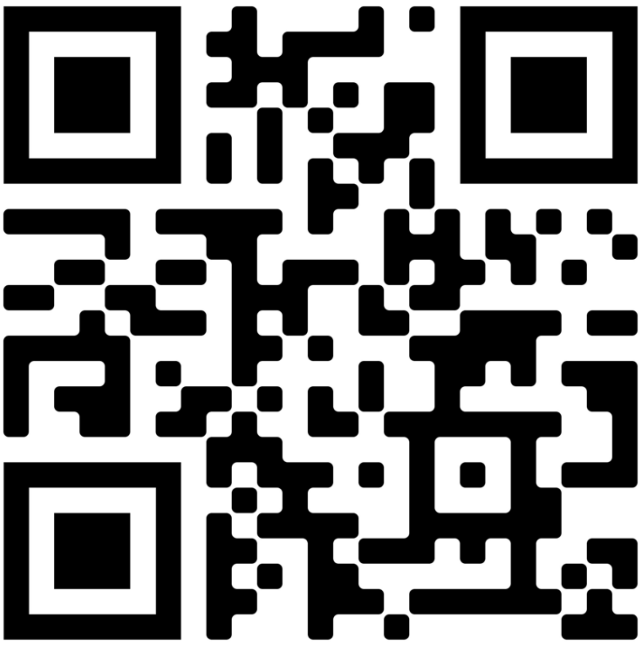
Esta abordagem permite manter a separação entre as operações de processamento realizadas no KNIME e a apresentação gráfica desenvolvida em Node-RED.

A disponibilização dos resultados através de um dashboard constitui uma funcionalidade complementar do projeto, contribuindo para uma interpretação mais acessível da informação e demonstrando a integração de diferentes tecnologias num processo de análise de dados.

4.2.9 – Vídeo de demonstração

De forma a complementar o relatório, foi realizado um vídeo de demonstração prática da implementação desenvolvida em KNIME. O vídeo apresenta a execução dos workflows, o tratamento e integração dos dados, o carregamento no PostgreSQL e a visualização dos resultados através do Node-RED.

O vídeo pode ser consultado através do seguinte QR Code.



Link: https://drive.google.com/file/d/1kZvIWJcJJ3nSVIgxceZ7-FFwFOZ6rWQL/view?usp=sharing

5- Conclusão e Trabalhos Futuros

O desenvolvimento deste projeto permitiu aplicar, de forma prática, os conceitos de Extração, Transformação e Carregamento (ETL), através da integração de diferentes fontes de dados relacionadas com os acidentes rodoviários ocorridos em Londres, durante o mês de agosto de 2025.

Ao longo do trabalho, foram desenvolvidos processos de preparação, limpeza, validação e transformação dos dados, utilizando as ferramentas Pentaho Kettle e KNIME. A informação relativa aos acidentes e veículos foi complementada com dados meteorológicos históricos, obtidos através da API Open-Meteo, permitindo relacionar diferentes conjuntos de informação.

Como resultado, foi possível processar e integrar 1701 registos de acidentes e 3120 registos de veículos, armazenando os dados em bases de dados relacionais e disponibilizando estatísticas para análise. Adicionalmente, a implementação em Pentaho Kettle incluiu a automatização do processo através de um Job, a geração de ficheiros JSON, o envio automático de estatísticas por correio eletrónico e a criação de um dashboard no Node-RED.

Durante o desenvolvimento, surgiram alguns desafios, nomeadamente na preparação de datasets com um elevado volume de informação, na integração de dados provenientes de diferentes fontes e na otimização das consultas à API meteorológica. A resolução destas dificuldades permitiu aprofundar conhecimentos sobre processos ETL, integração de sistemas e automatização de tarefas.

Relativamente a trabalhos futuros, seria interessante alargar a análise a outros períodos e regiões, permitindo realizar comparações temporais e geográficas. Poderia também ser implementada uma atualização automática e periódica dos dados, bem como o desenvolvimento de novas visualizações e indicadores estatísticos.

Em suma, considera-se que os objetivos propostos foram alcançados, tendo sido desenvolvida uma solução capaz de integrar, transformar, armazenar e apresentar informação proveniente de diferentes fontes, demonstrando a importância dos processos ETL na organização e análise de dados.

6- Referências Bibliográficas

Department for Transport. (2026). *Road Safety Open Data*. GOV.UK. Disponível em: https://www.gov.uk/government/statistical-data-sets/road-safety-open-data

Open-Meteo. (s.d.). *Historical Weather API — Documentation*. Disponível em: https://open-meteo.com/en/docs/historical-weather-api

Pentaho. (s.d.). *Pentaho Data Integration 10.2 — Documentation*. Disponível em: https://docs.pentaho.com/pdia-data-integration/10.2-data-integration

KNIME. (s.d.). *KNIME Analytics Platform — User Guide*. Disponível em: https://docs.knime.com/ap/latest/analytics_platform_user_guide/

Node-RED. (s.d.). *Node-RED User Guide*. Disponível em: https://nodered.org/docs/user-guide/

FlowFuse. (s.d.). *Node-RED Dashboard 2.0 — Documentation*. Disponível em: https://dashboard.flowfuse.com/

Data de consulta: 22 de outubro de 2026.